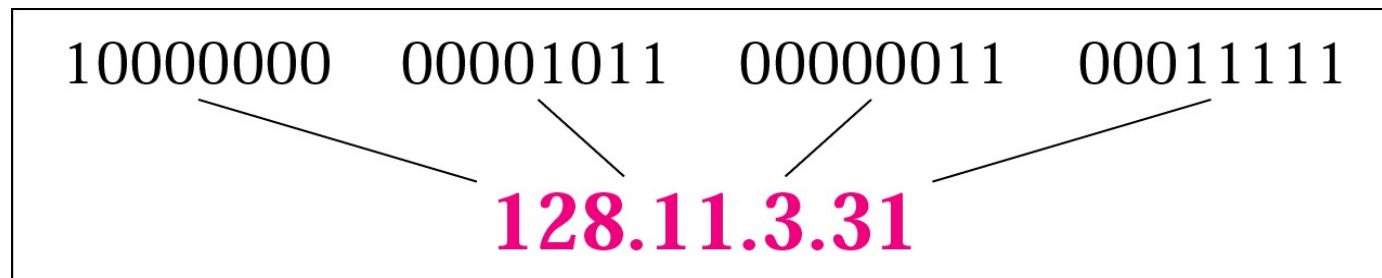

NETWORK LAYER

- IP Addresses
 - IPV4/IPV6
 - Subnetting
 - Supernetting
 - Congestion control
 - Address Mapping : ARP,RARP,ICMP,DHCP
-

IPv4 Addresses

- An IP address is a 32-bit address that uniquely and universally defines the connection of a device
- An Address space is the total no. of addresses used by the protocol
- The address space of IPv4 is 2^{32} or 4,294,967,296
- **Notations**
- Binary notation: 01110101 10010101 00011101 00000010
- Dotted-decimal notation: 117.149.29.2



Example

Change the following IP addresses from binary notation to dotted-decimal notation.

- a. 10000001 00001011 00001011 11101111
- b. 11111001 10011011 11111011 00001111

We replace each group of 8 bits with its equivalent decimal number and add dots for separation:

- a. 129.11.11.239
 - b. 249.155.251.15
-

Classful addressing

- In classful addressing, the address space is divided into five classes: A, B, C, D, E
- A new architecture, called classless addressing was introduced in the mid-1990s

	First byte	Second byte	Third byte	Fourth byte
Class A	0			
Class B	10			
Class C	110			
Class D	1110			
Class E	1111			

a. Binary notation

	First byte	Second byte	Third byte	Fourth byte
Class A	0–127			
Class B	128–191			
Class C	192–223			
Class D	224–239			
Class E	240–255			

b. Dotted-decimal notation

Classful Addressing: Example

- Find the class of each address.
 - a. 00000001 00001011 00001011 11101111
 - b. 11000001 10000011 00011011 11111111
 - c. 14.23.120.8
 - d. 252.5.15.111
 - Solution
 - a. The first bit is 0. This is a class A address.
 - b. The first 2 bits are 1; the third bit is 0. This is a class C address.
 - c. The first byte is 14; the class is A.
 - d. The first byte is 252; the class is E.
-

Classes and Blocks

- In classful addressing, a large part of the available addresses were wasted

<i>Class</i>	<i>Number of Blocks</i>	<i>Block Size</i>	<i>Application</i>
A	128	16,777,216	Unicast
B	16,384	65,536	Unicast
C	2,097,152	256	Unicast
D	1	268,435,456	Multicast
E	1	268,435,456	Reserved

Class A : large organizations

Class B : mid size organizations

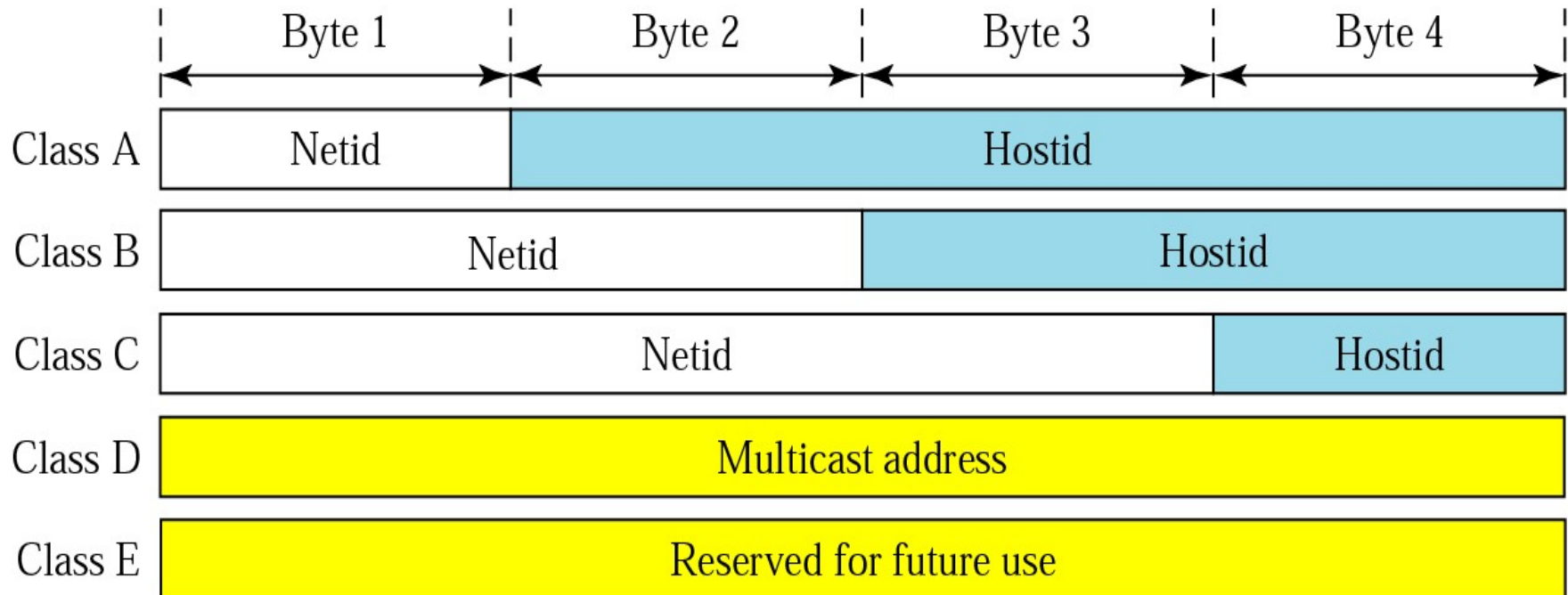
Class C: small organizations

Class D : multicasting

Class E : reserved for future use

Netid and Hostid

- IP address in classes A, B, and C is divided into **netid** and **hostid**
- **Netid: determines the network address**
- **Hostid : determines the host connected to that**



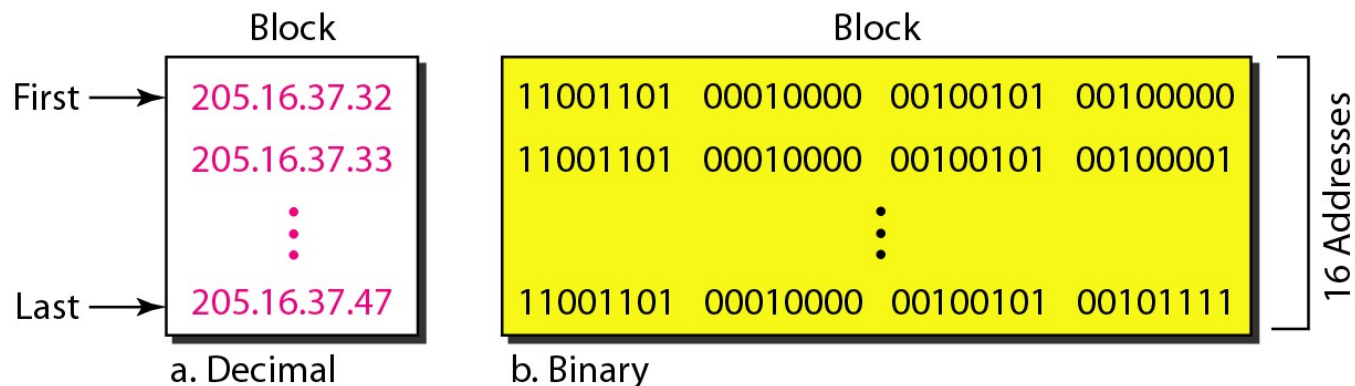
Mask: Default Mask

- The length of the netid and hostid is predetermined in classful addressing
- **Default masking**
- CIDR (Classless Interdomain Routing) notation

<i>Class</i>	<i>Binary</i>	<i>Dotted-Decimal</i>	<i>CIDR</i>
A	11111111 00000000 00000000 00000000	255 .0.0.0	/8
B	11111111 11111111 00000000 00000000	255.255 .0.0	/16
C	11111111 11111111 11111111 00000000	255.255.255 .0	/24

Classless addressing

- Classful addressing has created many problems
- Many ISPs and service users need more addresses
- Idea is to have variable-length blocks that belong to no class
- Three restrictions on classless address blocks;
 - The addresses in a block must be contiguous, one after another
 - The number of addresses in a block must be a power of 2
 - The first address must be evenly divisible by the number of addresses



Mask and Address Blocks

- In IPv4 addressing, a block of addresses can be defined as $x.y.z.t /n$ in which $x.y.z.t$ defines one of the addresses and the $/n$ defines the mask.
- The first address in the block can be found by setting the rightmost $32 - n$ bits to 0s
- The last address in the block can be found by setting the rightmost $32 - n$ bits to 1s
- The number of addresses in the block can be found by using the formula 2^{32-n}

Classless addressing

- Consider a block of addresses
 - 205.16.37.32
 - 205.16.37.33
 -
 - 205.16.37.47
-
- Total addresses = 16
 - contiguous
 - First address is divisible by 16
-

Example

- A block of addresses is granted to an organization ,one of the address is 205.16.37.39/28.What is the first address and last address in the block?

The binary representation is

1100110 00010000 00100101 00100111

- If we set 32 – 28 rightmost bits to 0, we get

– 11001101 00010000 00100101 00100000

→ 205.16.37.32 (First address)

- If we set 32 – 28 rightmost bits to 1, we get

– 11001101 00010000 00100101 00101111

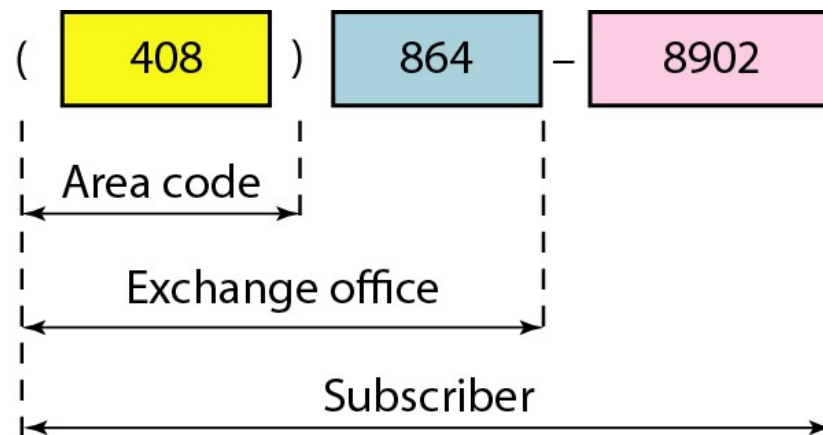
→ 205.16.37.47 (Last address)

- The value of n is 28, which means that number of addresses is 2^{32-28} or 16

Network Address

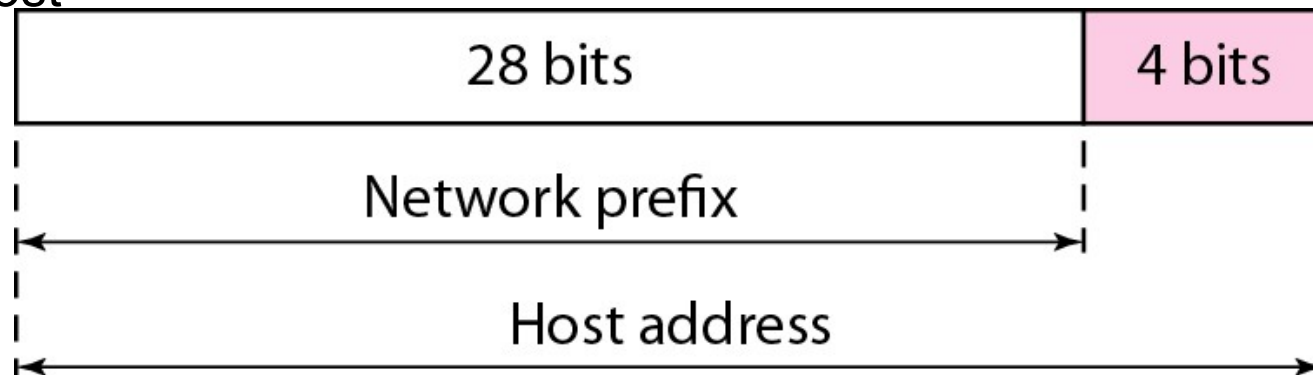
- The first address in a block is normally not assigned to any device; it is used as the network address that represents the organization to the rest of the world

Hierarchy



Two-Level Hierarchy: No Subnetting

- Each address in the block can be considered as a two-level hierarchical structure:
the leftmost n bits (prefix) define the network; the rightmost $32 - n$ bits define the host



Subnetting

- Divide a large block of addresses into several contiguous groups and assign each group to smaller networks called subnets
 - Increase the number of 1s in the mask
-

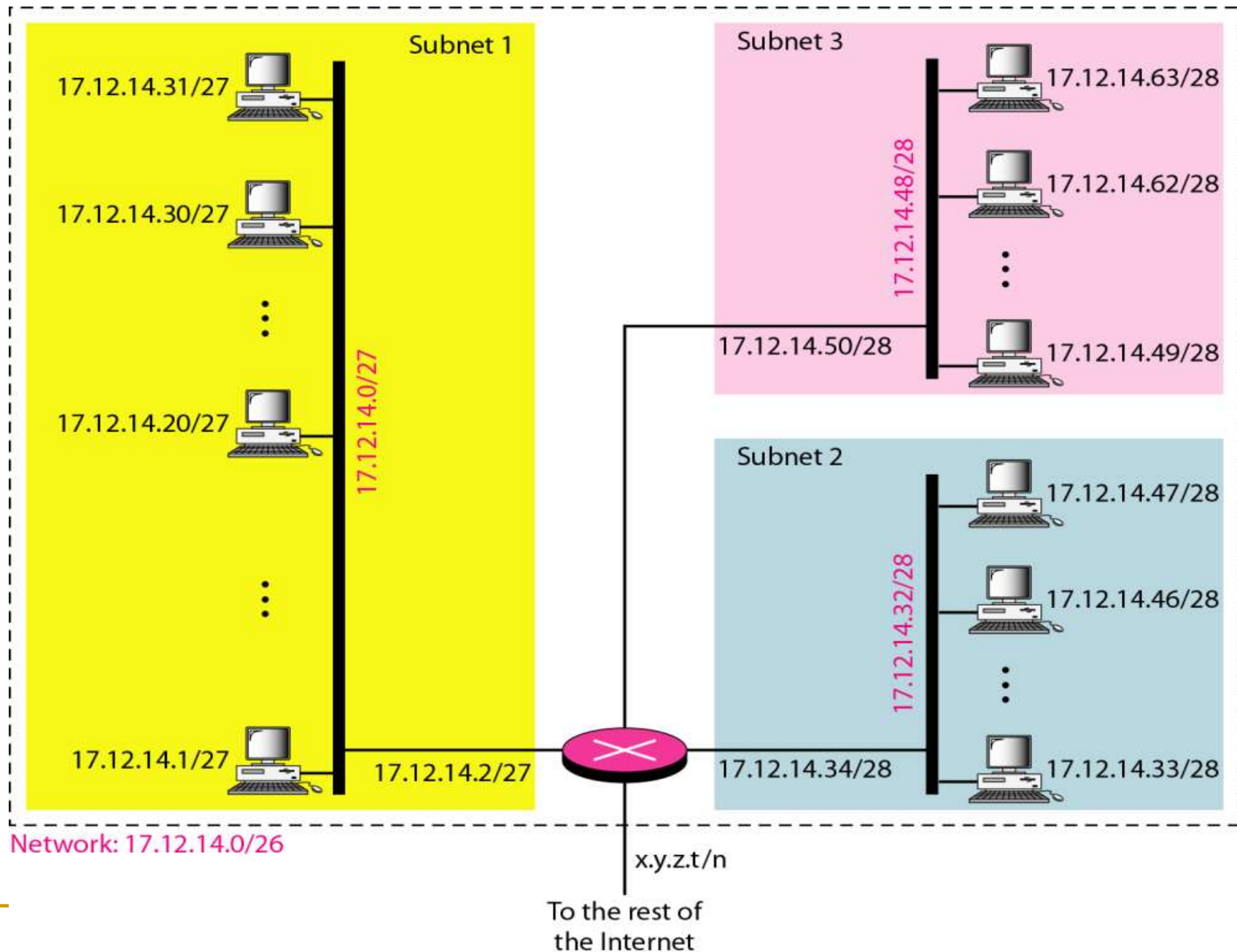
Subnets



A campus network consisting of LANs for various departments.

- Subnetting :Example
- An organization is given a block which contains 64 addresses. The organization has 3 offices and needs to divide the addresses in to three blocks of 32,16 &16 addresses
- First subnet mask n_1
- $32 - n_1 = 5$
- $n_1 = 27$
- $32 - n_2 = 4$
- $n_2 = 28$

Three-Levels of Hierarchy: Subnetting



Default subnet mask

Class	Default Subnet Mask	Address Range
A	255.0.0.0	1.0.0.0 – 126.255.255.255
B	255.255.0.0	128.0.0.0 – 191.255.255.255
C	255.255.255.0	192.0.0.0 – 223.255.255.255

A subnet mask is like an IP address that identifies the network and host parts within the original IP address. This identification helps establish specific routes for sending the data to a particular destination. Thus, the subnet mask serves as a tool that the network routers use to match the data packet's IP address with the destination's sub-network.

ANDING With Custom subnet masks

When you take a single network such as 192.100.10.0 and divide it into five smaller networks (192.100.10.16, 192.100.10.32, 192.100.10.48, 192.100.10.64, 192.100.10.80) the outside world still sees the network as 192.100.10.0, but the internal computers and routers see five smaller subnetworks. Each independent of the other. This can only be accomplished by using a custom subnet mask. A custom subnet mask borrows bits from the host portion of the address to create a subnetwork address between the network and host portions of an IP address. In this example each range has 14 usable addresses in it. The computer must still AND the IP address against the custom subnet mask to see what the network portion is and which subnetwork it belongs to.

IP Address: 192 . 100 . 10 . 0
Custom Subnet Mask: 255.255.255.240

Address Ranges: 192.10.10.0 to 192.100.10.15
192.100.10.16 to 192.100.10.31
192.100.10.32 to 192.100.10.47 (Range in the sample below)
192.100.10.48 to 192.100.10.63
192.100.10.64 to 192.100.10.79
192.100.10.80 to 192.100.10.95
192.100.10.96 to 192.100.10.111
192.100.10.112 to 192.100.10.127
192.100.10.128 to 192.100.10.143
192.100.10.144 to 192.100.10.159
192.100.10.160 to 192.100.10.175
192.100.10.176 to 192.100.10.191
192.100.10.192 to 192.100.10.207
192.100.10.208 to 192.100.10.223
192.100.10.224 to 192.100.10.239
192.100.10.240 to 192.100.10.255

	Network	Sub Network	Host	
IP Address:	1 1 0 0 0 0 0 0 . 0 1 1 0 0 1 0 0 . 0 0 0 0 1 0 1 0	0 0 1 0	0 0 0 1	(192 . 100 . 10 . 33)
Custom Subnet Mask:	1 1 1 1 1 1 1 1 . 0 1 1 1 1 1 1 1 . 1 1 1 1 1 1 1 1	1 1 1 1	0 0 0 0	(255 . 255 . 255 . 240)
AND:	1 1 0 0 0 0 0 0 . 0 1 1 0 0 1 0 0 . 0 0 0 0 1 0 1 0	0 0 1 0	0 0 0 0	(192 . 100 . 10 . 32)

Four bits borrowed from the host
portion of the address for the
custom subnet mask.

The ANDING process of the four borrowed bits
shows which range of IP addresses this
particular address will fall into.

Supernetting

- Combine several class C blocks to create a larger range of addresses
 - Decrease the number of 1s in the mask (/24 → /22 for C addresses)
-

Supernetting

3. Example — Combine 4 Class C Networks

Let's say we have these 4 contiguous Class C networks:

Network	Prefix	Range
192.168.4.0	/24	192.168.4.0 – 192.168.4.255
192.168.5.0	/24	192.168.5.0 – 192.168.5.255
192.168.6.0	/24	192.168.6.0 – 192.168.6.255
192.168.7.0	/24	192.168.7.0 – 192.168.7.255

12
34

4. Combine in Binary

Look at the **third octet** (the one that changes):

ini

4 = 00000100

5 = 00000101

6 = 00000110

7 = 00000111

The **first 6 bits** (000001) are the same.

The **last 2 bits** vary (00, 01, 10, 11).



So, **common prefix length = 22 bits**

Hence, the combined (supernet) address = **192.168.4.0/22**

Advantages of Supernetting:

- Control and reduce network traffic
 - Helpful to solve the problem of lacking IP addresses
 - Minimizes the routing table i.e, it cannot cover a different area of the network when combined and all the networks should be in the same class and all IP should be contiguous
-

NAT

- Network Address Translation (NAT) is a service that operates on a router or edge platform to connect private networks to public networks like the internet. NAT is often implemented at the WAN edge router to enable internet access in core, campus, branch, and co location sites.
 - With NAT, an organization needs one IP address or one limited public IP address to represent an entire group of devices as they connect outside their network. Port Address Translation (PAT) enables one single IP to be shared by multiple hosts using IP and port address translation.
-

Figure 19.10 *A NAT
implementation*

Site using private addresses

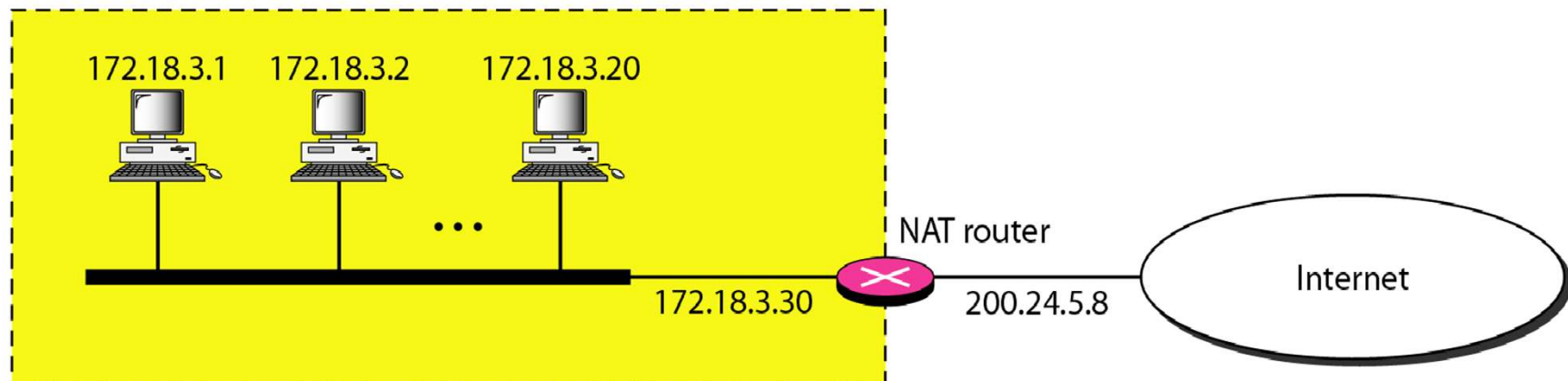
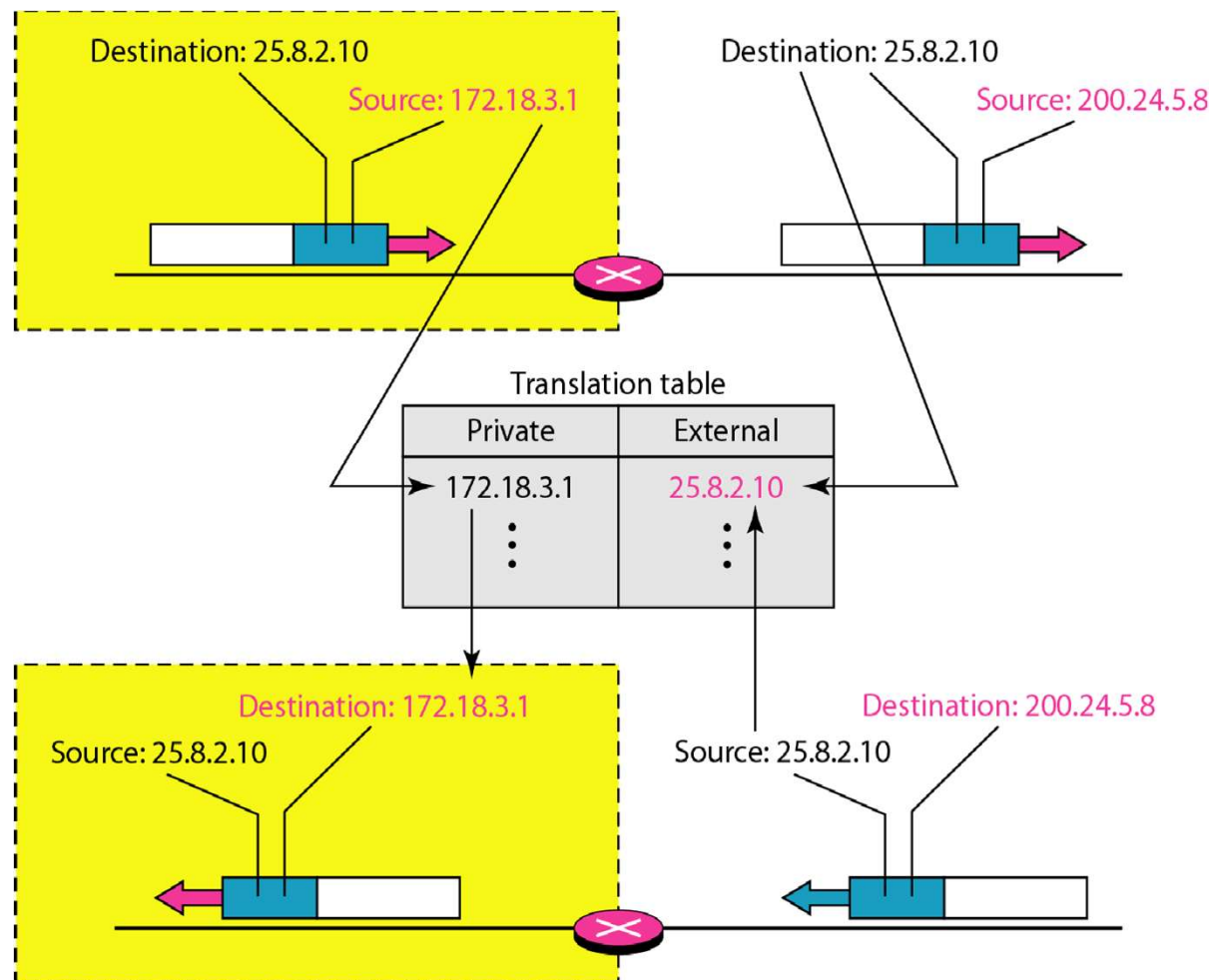


Figure 19.12 *NAT address translation*

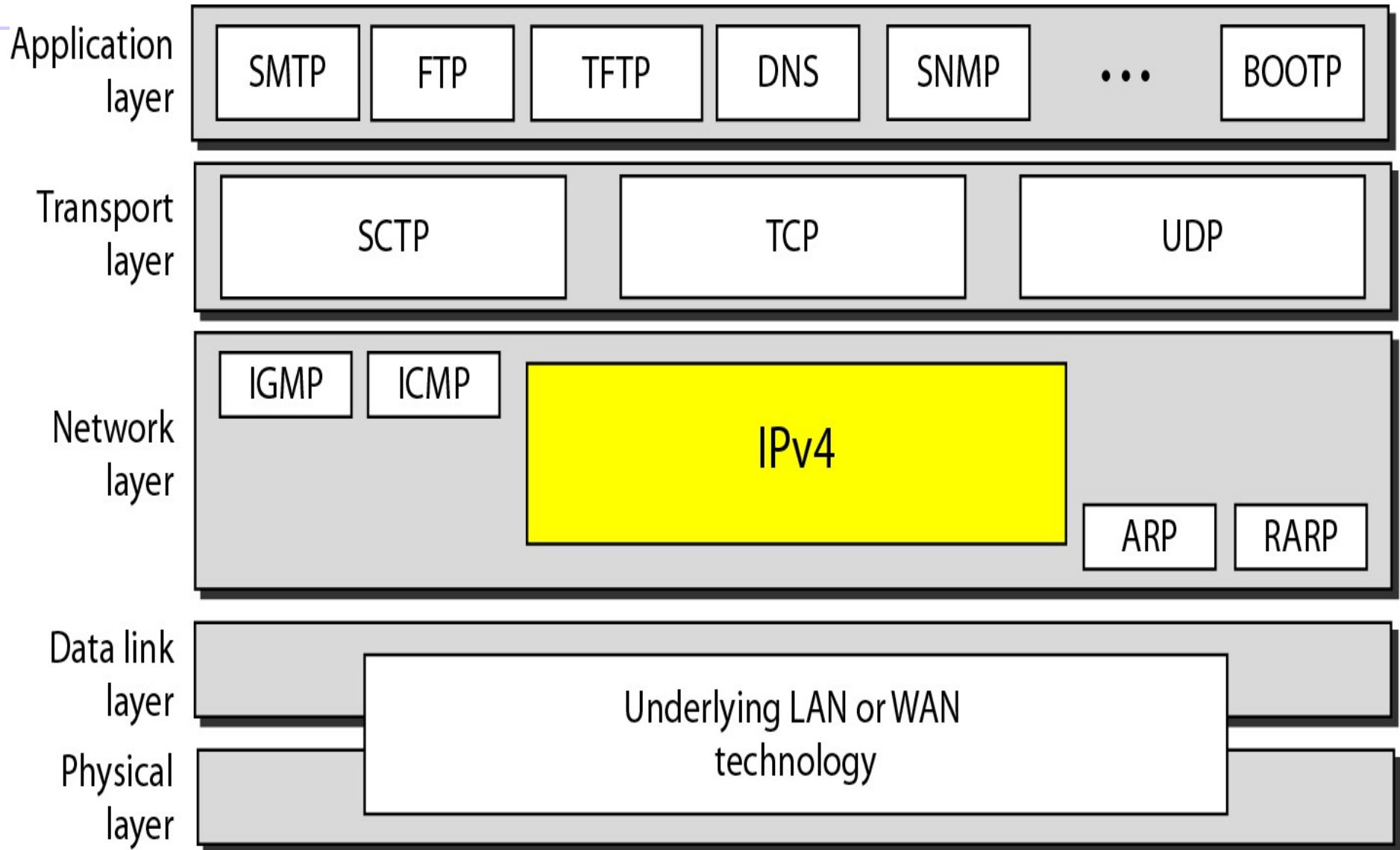


IPV4(Internet Protocol version 4)

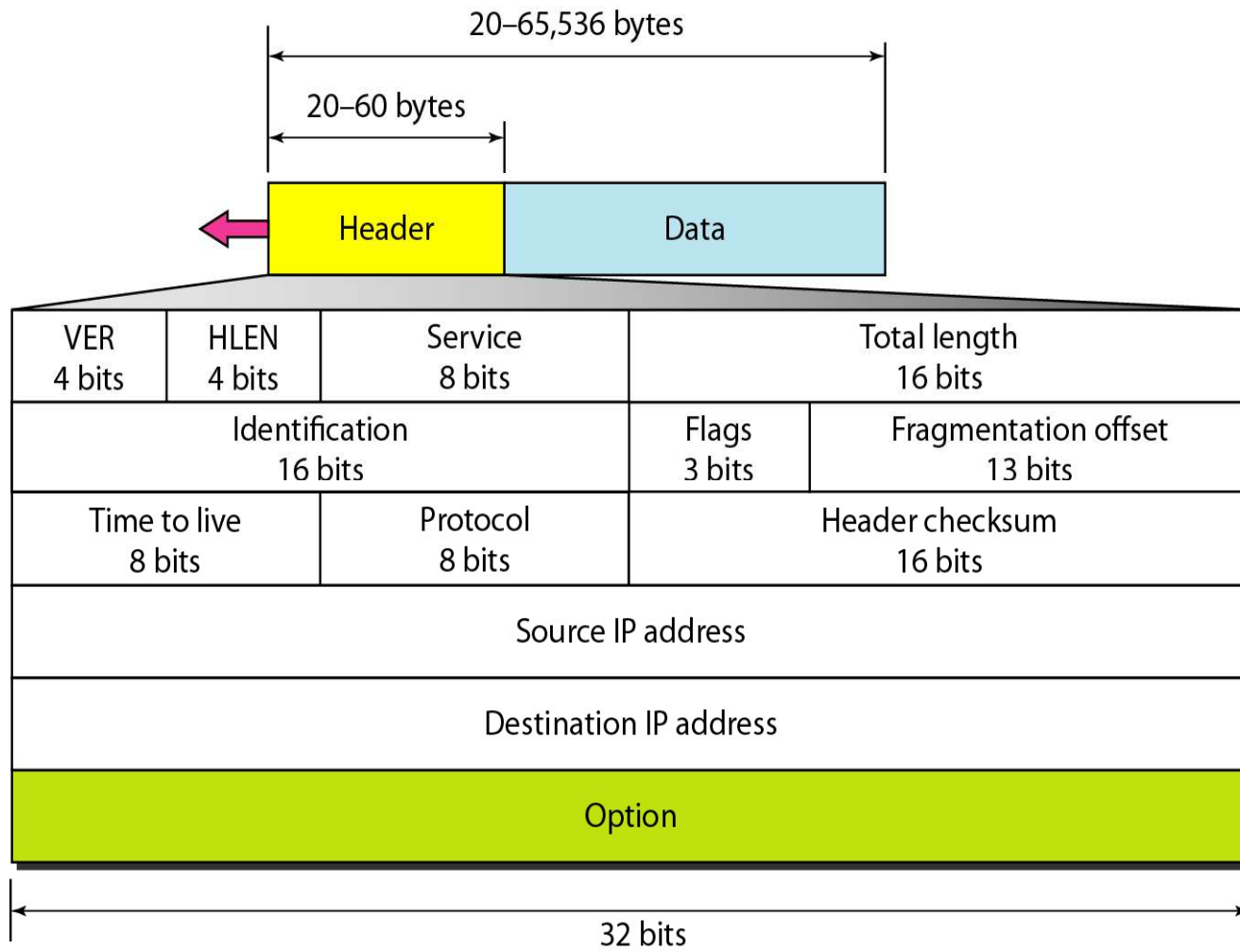
The Internet Protocol version 4 (IPv4) is the delivery mechanism used by the TCP/IP protocols

- *Connectionless and unreliable protocol*
 - *A best effort delivery service*
 - *For reliability IPV4 must be paired with a protocol such as TCP*
 - *Packets in the IPV4 layers are called Datagrams*
-

POSITION OF IPV4 in TCP/IP

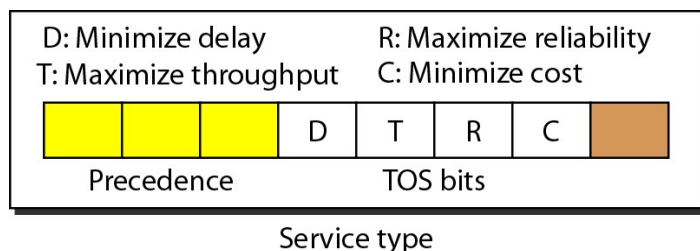


IPV4 :Datagram format



-
- **Version:(4 bits):** The Version field keeps track of which version of the protocol the datagram belongs to.
 - **Internet Header Length(IHL 4 bits):** IHL, is provided to tell how long the header is, in 32-bit words. The minimum value is 5, which applies when no options are present. The maximum value of this 4-bit field is 15, which limits the header to 60 bytes, and thus the Options field to 40 bytes.
 - **Type-of-Service (8 bits):** The Type of service field is one of the few fields that has changed its meaning (slightly) over the years. It was and is still intended to distinguish between different classes of service. Various combinations of reliability and speed are possible. For digitized voice, fast delivery beats accurate delivery. For file transfer, error-free transmission is more important than fast transmission.
-

- Originally, the 6-bit field contained (from left to right), a three-bit Precedence field and three flags, D, T, and R. The Precedence field was a priority, from 0 (normal) to 7 (network control packet). The three flag bits allowed the host to specify what it cared most about from the set {Delay, Throughput, Reliability}.



<i>TOS Bits</i>	<i>Description</i>
0000	Normal (default)
0001	Minimize cost
0010	Maximize reliability
0100	Maximize throughput
1000	Minimize delay

- Total length(16 bits):** The Total length includes everything in the datagram—both header and data. The maximum length is 65,535 bytes. At present, this upper limit is tolerable, but with future gigabit networks, larger data grams may be needed.

Default TOS for Applications

<i>Protocol</i>	<i>TOS Bits</i>	<i>Description</i>
ICMP	0000	Normal
BOOTP	0000	Normal
NNTP	0001	Minimize cost
IGP	0010	Maximize reliability
SNMP	0010	Maximize reliability
TELNET	1000	Minimize delay
FTP (data)	0100	Maximize throughput
FTP (control)	1000	Minimize delay
TFTP	1000	Minimize delay
SMTP (command)	1000	Minimize delay
SMTP (data)	0100	Maximize throughput
DNS (UDP query)	1000	Minimize delay
DNS (TCP query)	0000	Normal
DNS (zone)	0100	Maximize throughput

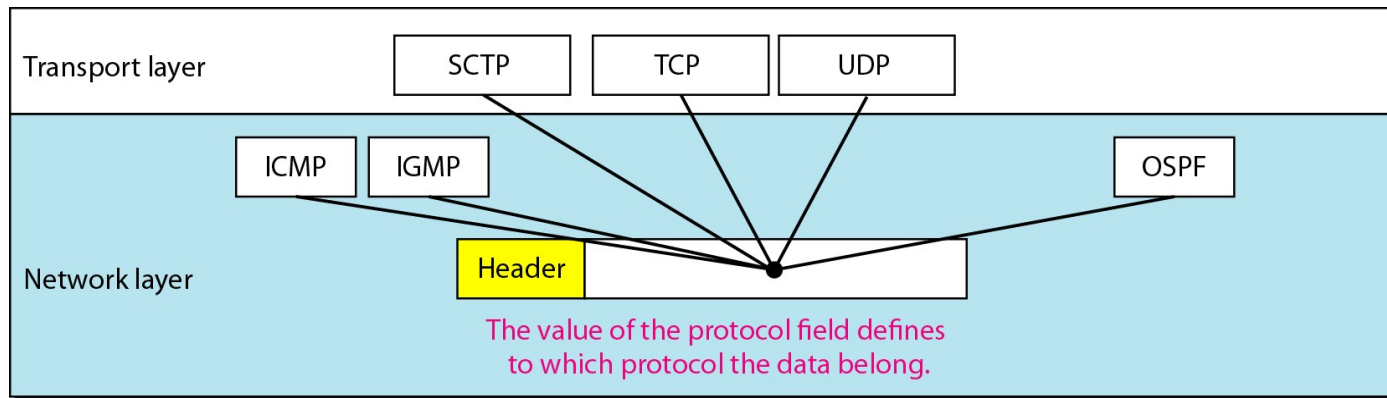
-
- **Identifier (16 bits):** To have a proper reassembling of fragments, The Identification field is needed to allow the destination host to determine which datagram a newly arrived fragment belongs to. **All the fragments of a datagram contain the same Identification value.**
 - **Flags(3 bits):** First one an **unused bit** Only two of the bits are currently defined: **MF(More Fragments)** ,**DF(Don't Fragment)**
MF(More Fragments) :When a receiving host sees a packet arrive with the MF = 1, it examines the Fragment Offset to see where this fragment is to be placed in the reconstructed packet.
Don't Fragment flag (DF):The Don't Fragment (DF) flag is a single bit in the Flag field that indicates that fragmentation of the packet is not allowed.
-

-
- **Fragment offset:** The Fragment offset tells where in the current datagram this fragment belongs.
 - **Time-to-Live (TTL) (8 bits):** The Time to live field is a counter used to limit packet lifetimes. It is supposed to count time in seconds, allowing a maximum lifetime of 255 sec. It must be decremented on each hop and is supposed to be decremented multiple times when queued for a long time in a router. In practice, it just counts hops. When it hits zero, the packet is discarded and a warning packet is sent back to the source host..
-

-
- **Protocol (8 bits):** The Protocol field tells it which transport process to give it to. TCP is one possibility, but so are UDP and some others.
 - **Header checksum (16 bits):** The Header checksum verifies the header only. Such a checksum is useful for detecting errors generated by bad memory words inside a router.
-

IPv4 Header

- Protocol field for higher-level protocol



<i>Value</i>	<i>Protocol</i>
1	ICMP
2	IGMP
6	TCP
17	UDP
89	OSPF

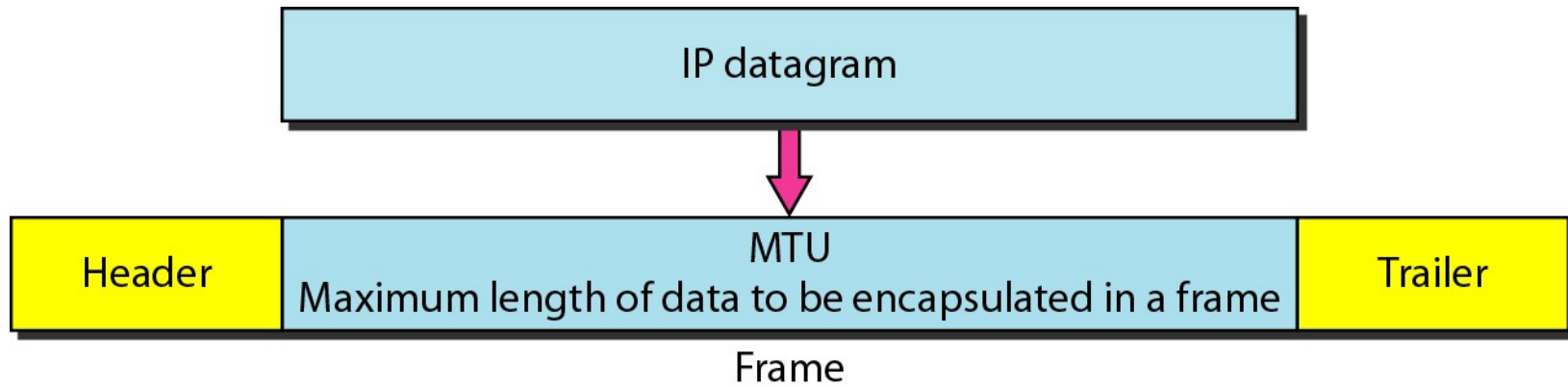
-
- **IP Destination Address (32 bits):** The IP Destination Address field contains a 32-bit binary value that represents the packet destination Network layer host address.
 - **IP Source Address (32 bits):** The IP Source Address field contains a 32-bit binary value that represents the packet source Network layer host address.
 - **Options (variable):** The Options field is padded out to a multiple of four bytes. Originally, five options were defined. The current complete list is now maintained on-line at www.iana.org/assignments/ip-parameters.
-

Option	Description
Security	Specifies how secret the datagram is
Strict source routing	Gives the complete path to be followed
Loose source routing	Gives a list of routers not to be missed
Record route	Makes each router append its IP address
Timestamp	Makes each router append its address and timestamp

- The **Security** option tells how secret the information is.
- The **Strict source routing** option gives the complete path from source to destination as a sequence of IP addresses. The datagram is required to follow that exact route.

-
- The **Loose source routing** option requires the packet to traverse the list of routers specified, and in the order specified, but it is allowed to pass through other routers on the way.
 - The **Record route** option tells the routers along the path to append their IP address to the option field. This allows system managers to track down bugs in the routing algorithms.
 - Finally, the **Timestamp** option is like the Record route option, except that in addition to recording its 32-bit IP address, each router also records a 32-bit timestamp. This option, too, is mostly for debugging routing algorithms.
-

MTU

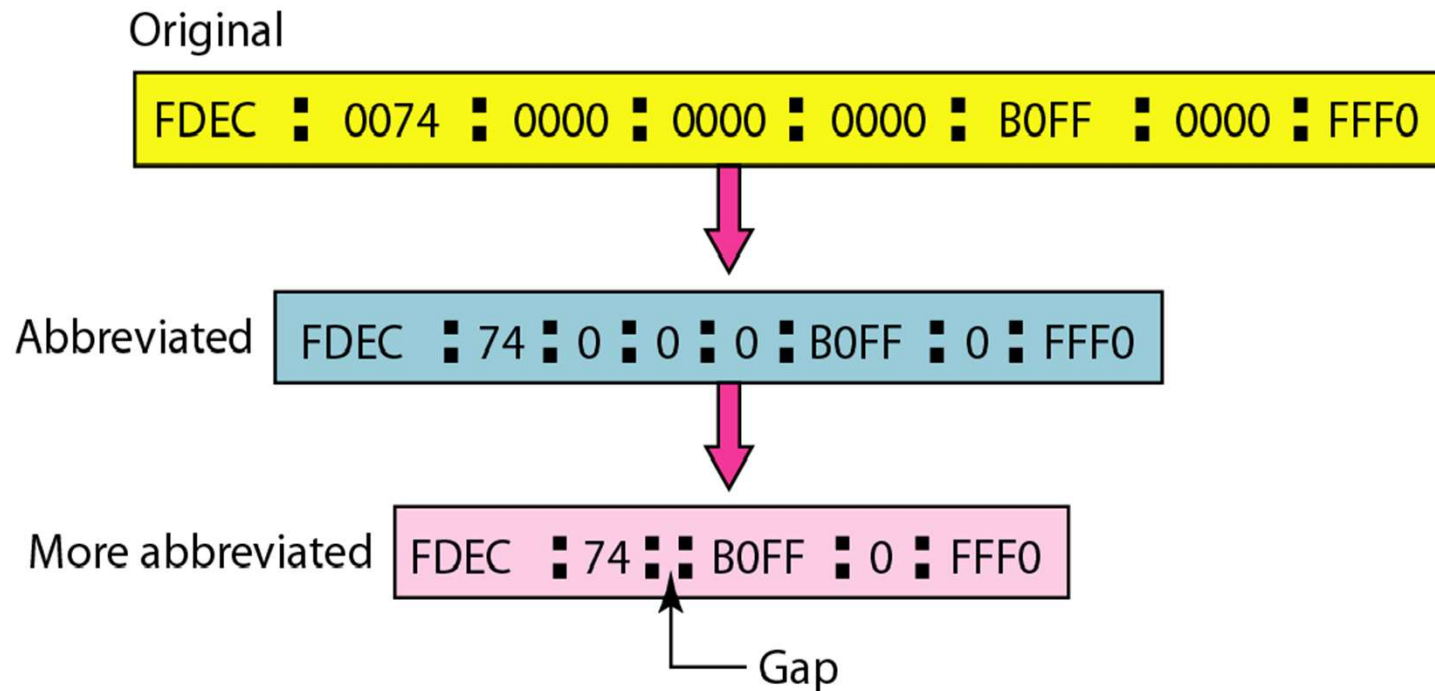


IPv6

IPv6 address

- The use of address space is inefficient
 - Minimum delay strategies and reservation of resources are required to accommodate real-time audio and video transmission
 - No security mechanism (encryption and authentication) is provided
 - IPv6 (IPng: Internetworking Protocol, next generation)
 - Larger address space (128 bits)
 - Better header format
 - New options
 - Allowance for extension
 - Support for resource allocation: flow label to enable the source to request special handling of the packet
 - Support for more security
-

Figure 19.15 *Abbreviated IPv6
addresses*



Example

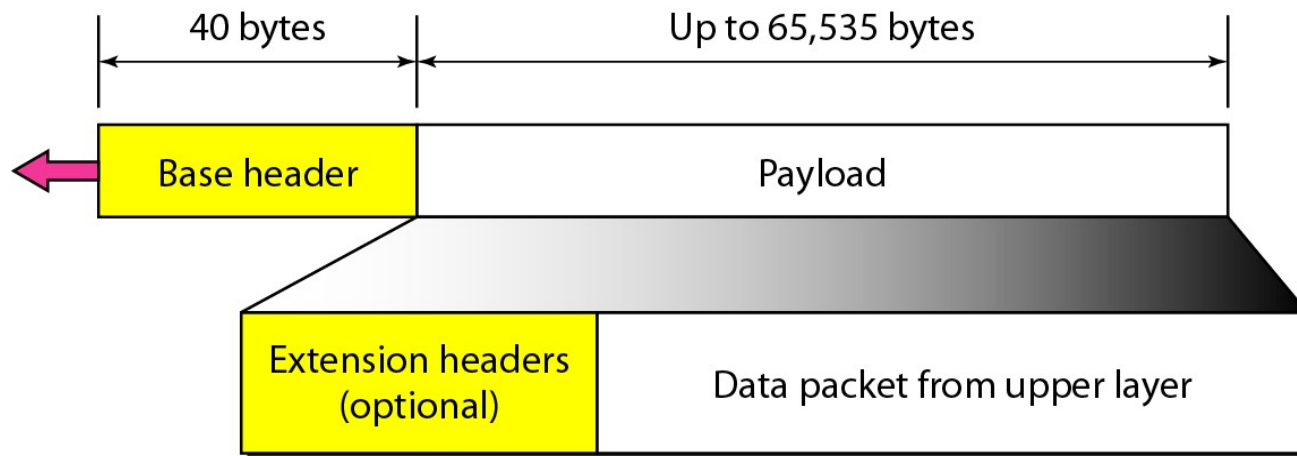
Expand the address 0:15::1:12:1213 to its original.

XXXX:XXXX:XXXX:XXXX:XXXX:XXXX:XXXX:XXXX
0: 15: : 1: 12:1213

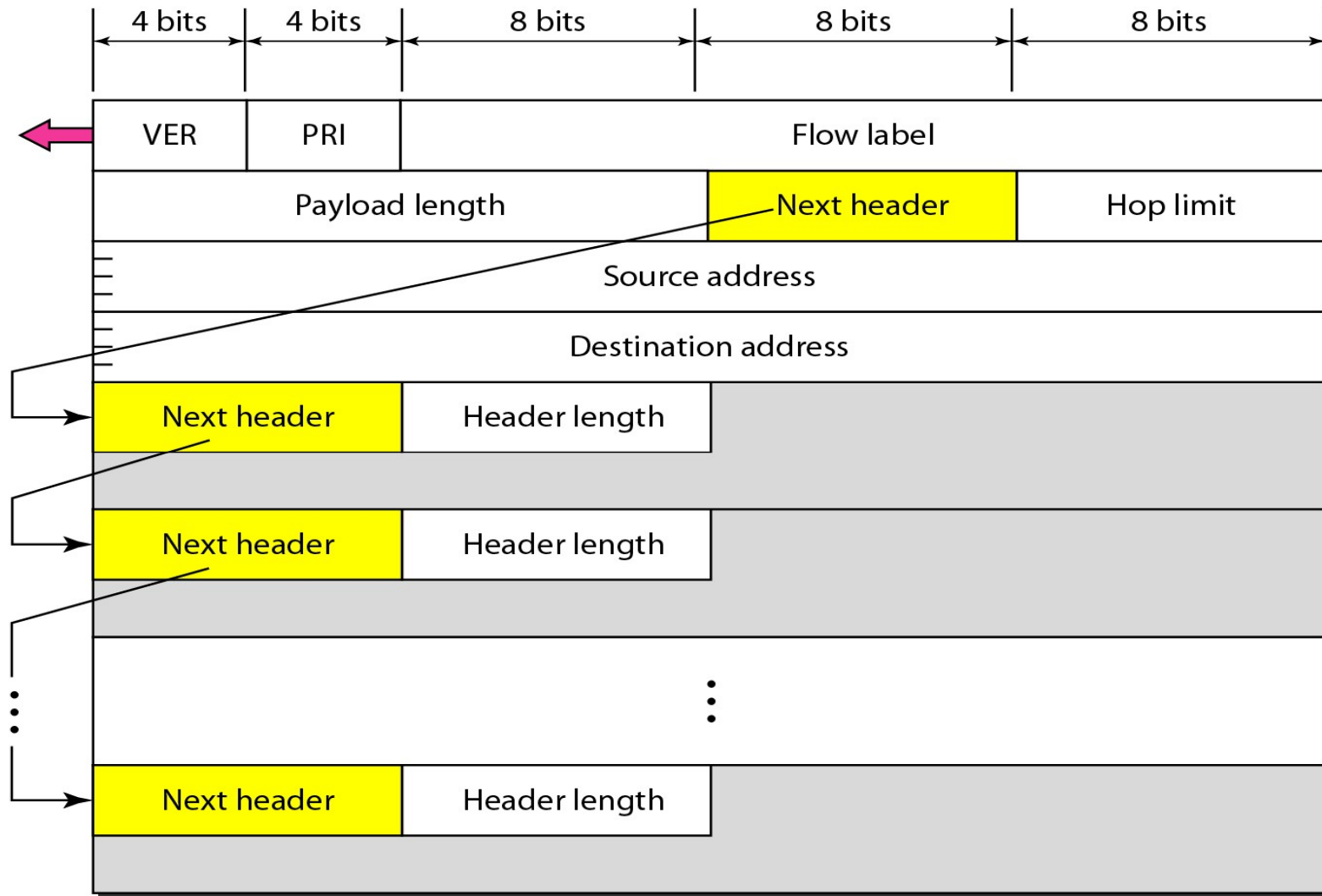
0000:0015:0000:0000:0000:0001:0012:1213

IPv6 Datagram

- IPv6 defines three types of addresses: unicast, anycast (a group of computers with the same prefix address), and multicast
- IPv6 datagram header and payload



IPv6 Datagram Format



IPv6 Header

- Version: IPv6
 - Priority (4 bits): the priority of the packet with respect to traffic congestion
 - Next header (8 bits): to define the header that follows the base header in the datagram
 - Hop limit: TTL in IPv4
 - Source address (16 bytes) and destination address (16 bytes): if source routing is used, the destination address field contains the address of the next router
-

Priority

- IPv6 divides traffic into two broad categories: congestion-controlled and noncongestion-controlled
- Congestion-controlled traffic

<i>Priority</i>	<i>Meaning</i>
0	No specific traffic
1	Background data
2	Unattended data traffic
3	Reserved
4	Attended bulk data traffic
5	Reserved
6	Interactive traffic
7	Control traffic

- Noncongestion-controlled traffic

<i>Priority</i>	<i>Meaning</i>
8	Data with greatest redundancy
...	...
15	Data with least redundancy

Extension Headers

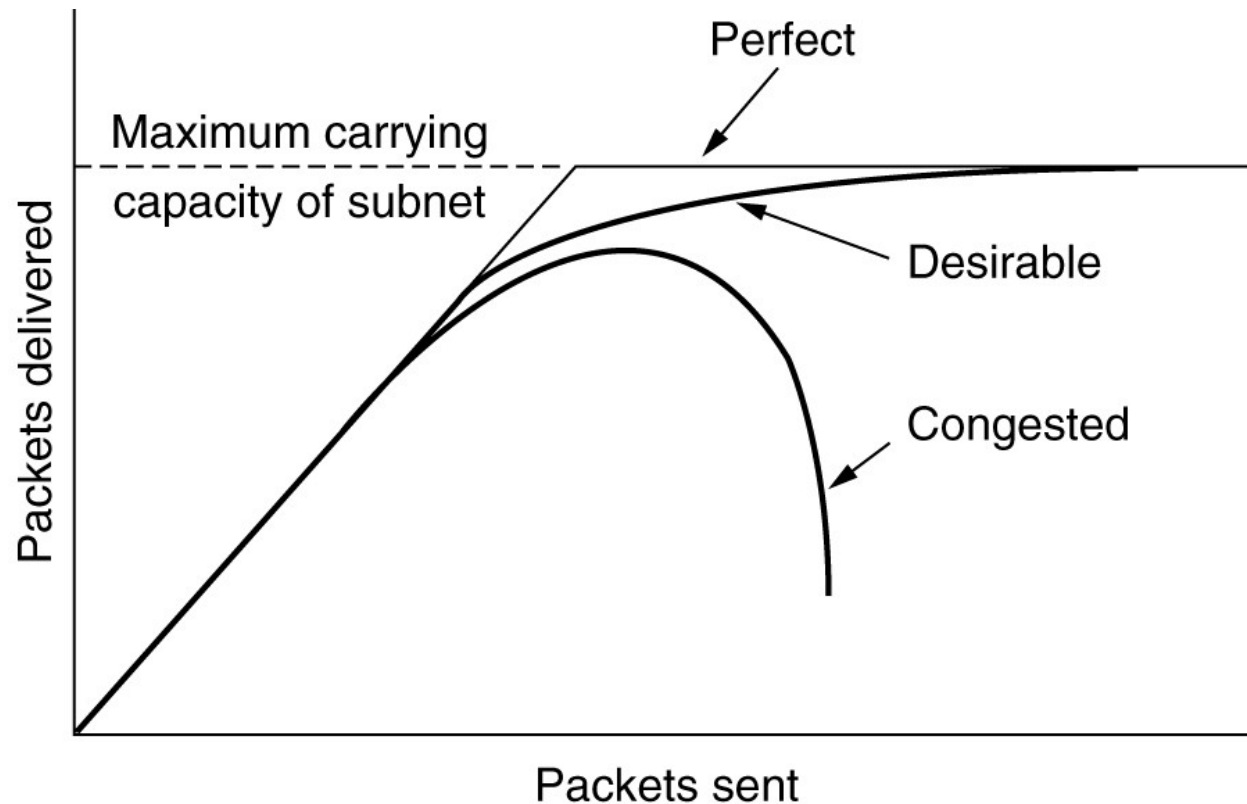
Extension header	Description
Hop-by-hop options	Miscellaneous information for routers
Destination options	Additional information for the destination
Routing	Loose list of routers to visit
Fragmentation	Management of datagram fragments
Authentication	Verification of the sender's identity
Encrypted security payload	Information about the encrypted contents

IPv6 extension headers.

IPv4/IPv6

IPv4	IPv6
IPv4 addresses are 32 bit length.	IPv6 addresses are 128 bit length.
IPv4 addresses are binary numbers represented in decimals.	IPv6 addresses are binary numbers represented in hexadecimals.
IPSec support is only optional.	Inbuilt IPSec support.
Fragmentation is done by sender and forwarding routers.	Fragmentation is done only by sender.
No packet flow identification.	Packet flow identification is available within the IPv6 header using the Flow Label field.
Checksum field is available in IPv4 header.	No checksum field in IPv6 header .
Options fields are available in IPv4 header .	No option fields, but IPv6 Extension headers are available

Congestion



When too much traffic is offered, congestion sets in and performance degrades sharply.

Factors that Cause Congestion

- Packet arrival rate exceeds the outgoing link capacity.
 - Insufficient memory to store arriving packets
 - Bursty traffic
 - Slow processor
-

Congestion Control vs Flow Control

- Congestion control is a global issue – involves every router and host within the subnet
- Flow control – scope is point-to-point; involves just sender and receiver.

Congestion Control Algorithms

- General Principles of Congestion Control
 - Congestion Prevention Policies
 - Congestion Control in Virtual-Circuit Subnets
 - Congestion Control in Datagram Subnets
-

General Principles of Congestion Control

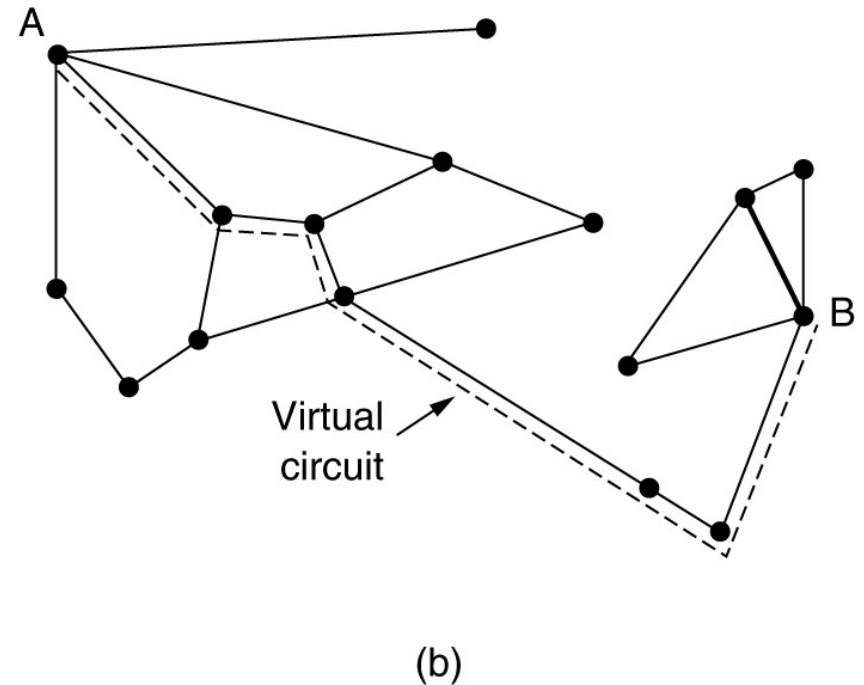
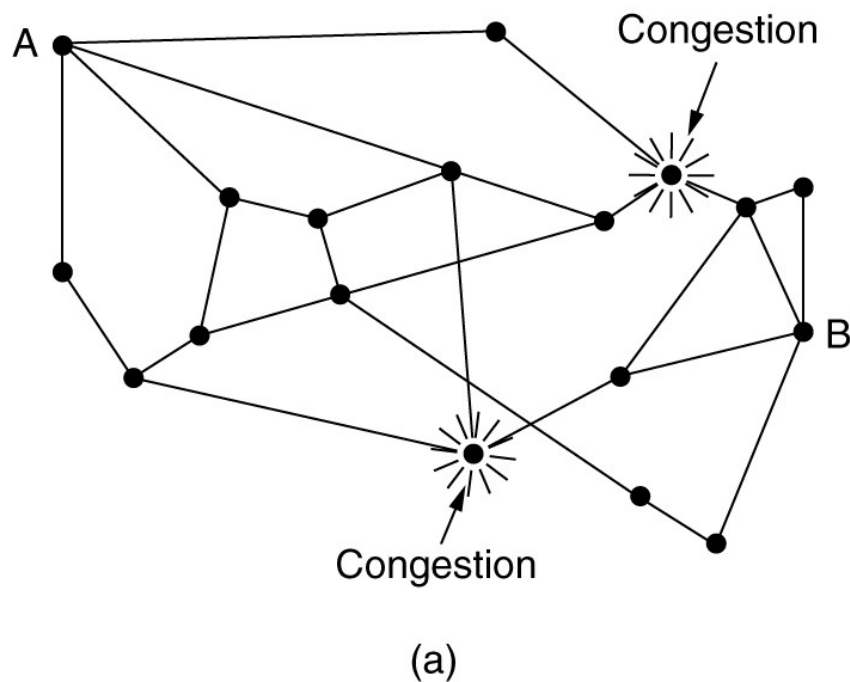
- Open loop & closed loop
 1. Open loop include when to accept & when to discard packets, which ones, and making scheduling decisions at various points in the network.
 2. Closed loop based on feedback loop
 - Monitor the system detect when and where congestion occurs.
 - Pass information to where action can be taken.
 - Adjust system operation to correct the problem.
-

Congestion Prevention Policies

Layer	Policies
Transport	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy • Timeout determination
Network	• Virtual circuits versus datagram inside the subnet • Packet queueing and service policy • Packet discard policy • Routing algorithm • Packet lifetime management
Data link	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy

Policies that affect congestion.

Congestion Control in Virtual-Circuit Subnets



(a) A congested subnet. (b) A redrawn subnet, eliminates congestion and a virtual circuit from A to B.

Congestion Control in Datagram Subnets

- Warning Bit
 - Choke packet
 - Hop-by-Hop Choke packet
 - Load Shedding
 - Jitter Control
-

Warning Bit

- A special bit in the packet header is set by the router to warn the source when congestion is detected.
- The bit is copied and piggy-backed on the ACK and sent to the sender.
- The sender monitors the number of ACK packets it receives with the warning bit set and adjusts its transmission rate accordingly.

Source based approach

- Warning bit
 - Output line in warning state
 - Warning bit set in header
 - Destination copies bit into next ack
 - Source cuts back traffic
 - Algorithm at source
 - As long as warning bits arrive: reduce traffic
 - Less warning bits: increase traffic
 - Problems
 - voluntary action of host!
 - correct source selected?
 - Used in
 - DecNet
 - Frame relay

Choke Packet

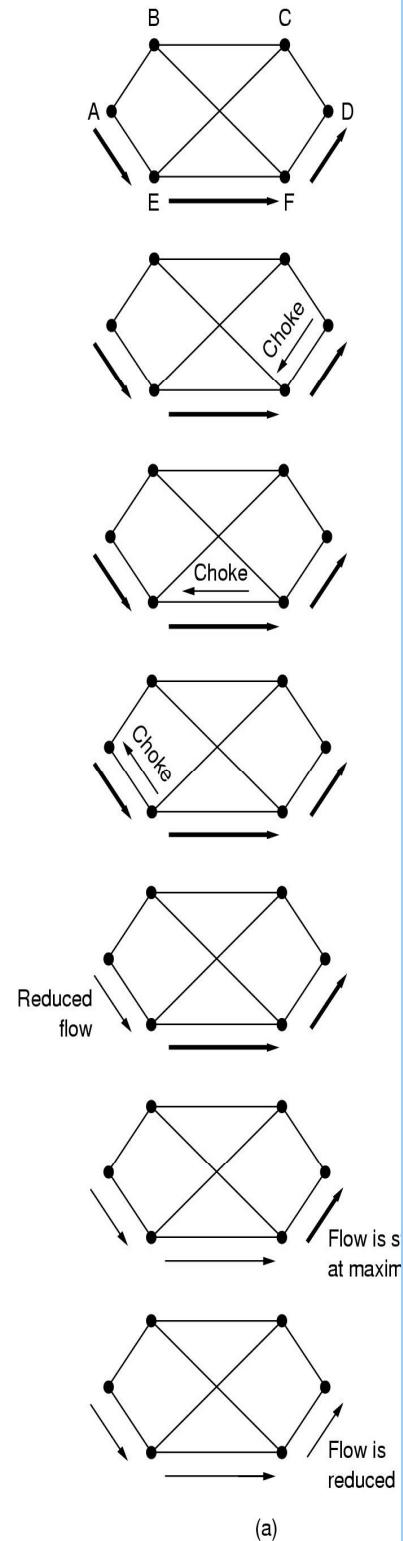
- A more direct way of telling the source to slow down.
 - A choke packet is a control packet generated at a congested node and transmitted to restrict traffic flow.
 - The source, on receiving the choke packet must reduce its transmission rate by a certain percentage.
 - An example of a choke packet is the ICMP Source Quench Packet.
-

Source based approach

- Choke packet
 - In case of overload: router sends choke packet to host causing the overload
 - Host receiving choke packet
 - reduces traffic to the specified destination
 - ignores choke packets for a fixed interval
 - new choke packets during next listening interval?
 - Yes: reduce traffic
 - No: increase traffic
 - Problems:
 - voluntary action of host!
 - correct host selected?

Source based approach

- Choke packets:
 - Example showing slow reaction
 - Solution: Hop-by-Hop choke packets



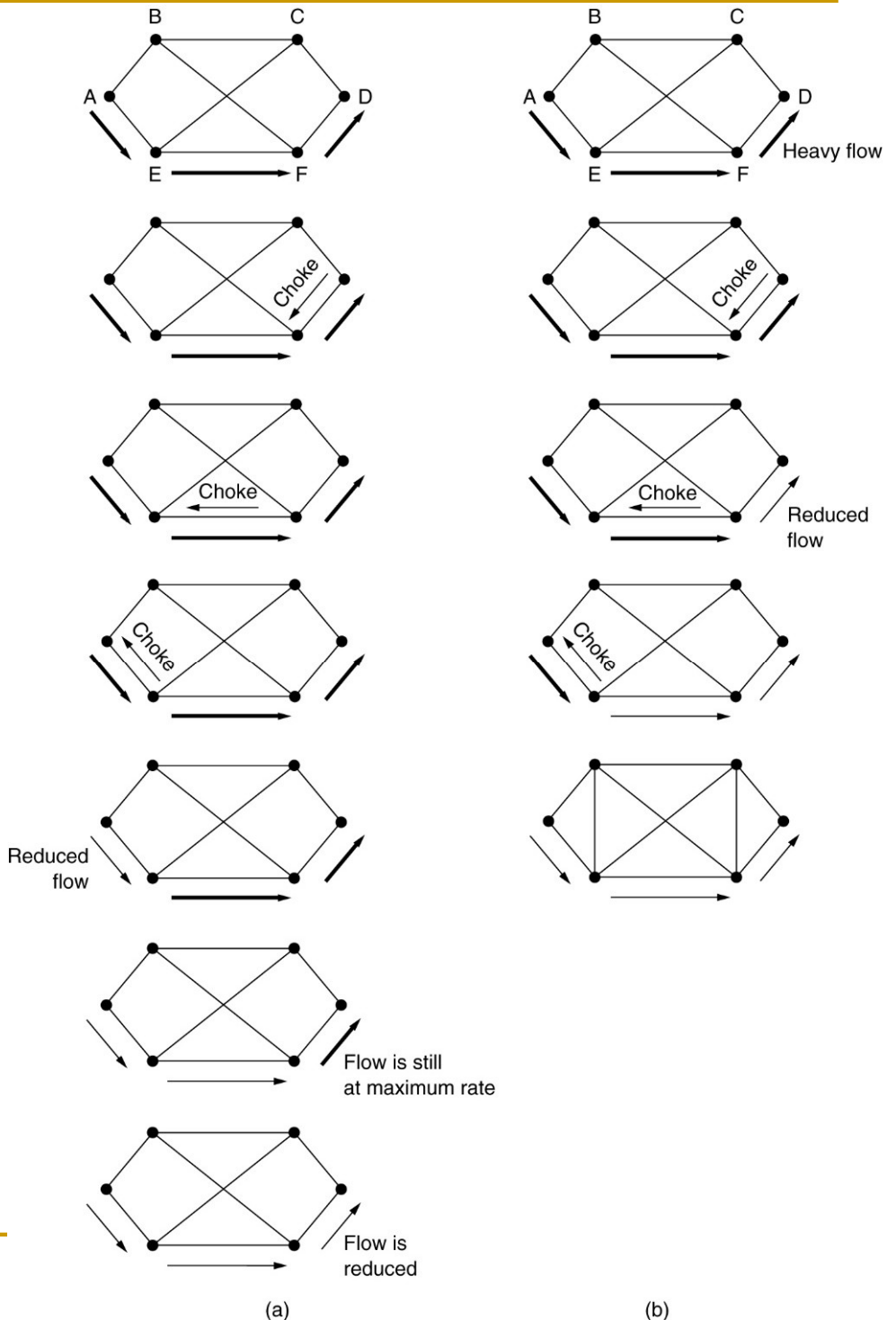
Hop-by-Hop Choke Packets

- Over long distances or at high speeds choke packets are not very effective.
 - A more efficient method is to send to choke packets hop-by-hop.
 - This requires each hop to reduce its transmission even before the choke packet arrive at the source.
-

Hop-by-Hop Choke Packets

(a) A choke packet that affects only the source.

(b) A choke packet that affects each hop it passes through.



Load Shedding

- When buffers become full, routers simply discard packets.
- Which packet is chosen to be the victim depends on the application and on the error strategy used in the data link layer.
- For a file transfer, for, e.g. cannot discard older packets since this will cause a gap in the received data.
 - For real-time voice or video it is probably better to throw away old data and keep new packets.
- Get the application to mark packets with discard priority.

Load shedding

- Throw away packets that cannot be handled!!
- Packet selection?
 - Random
 - Based on application
 - File transfer: discard new packet
 - Multimedia: discard old packet
 - Let sender indicate importance of packets
 - Low, high priority
 - Incentive to mark a packet with low priority
 - Price
 - Allow hosts to exceed agreed upon limits
- *Random early detection ...*

Load shedding

- Throw away packets that cannot be handled!!
- *Packet selection?*
- Random early detection
 - Discard packets before all buffer space is exhausted
 - Routers maintain running average of queue lengths
 - Select at random a packet
 - Inform source?
 - Send choke packet? → more load!!
 - No reporting
 - When does it work?
 - Source slows down when packets are lost

Random Early Discard (RED)

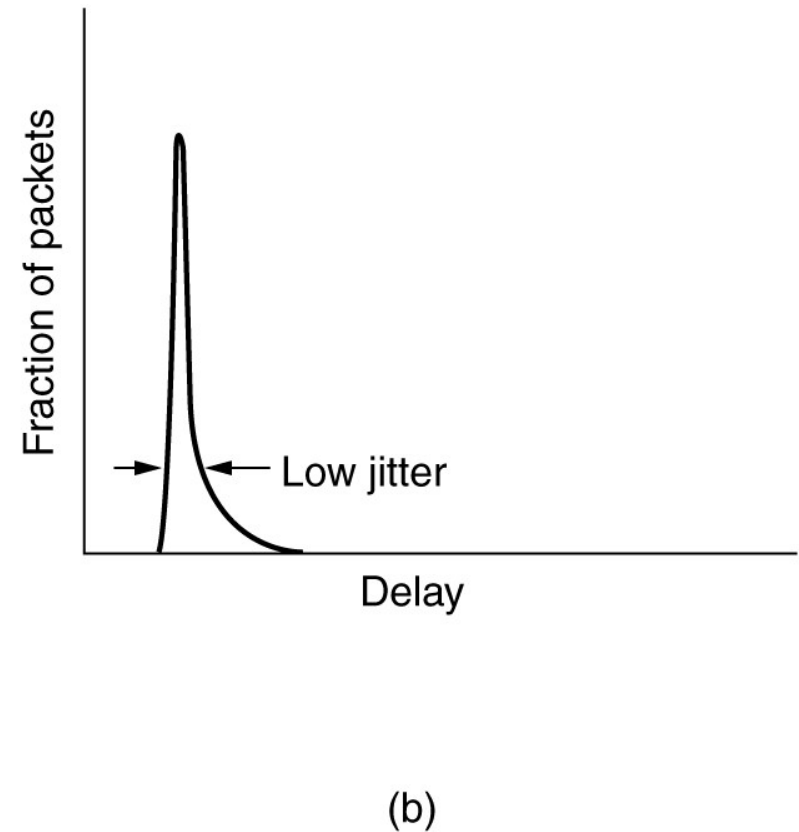
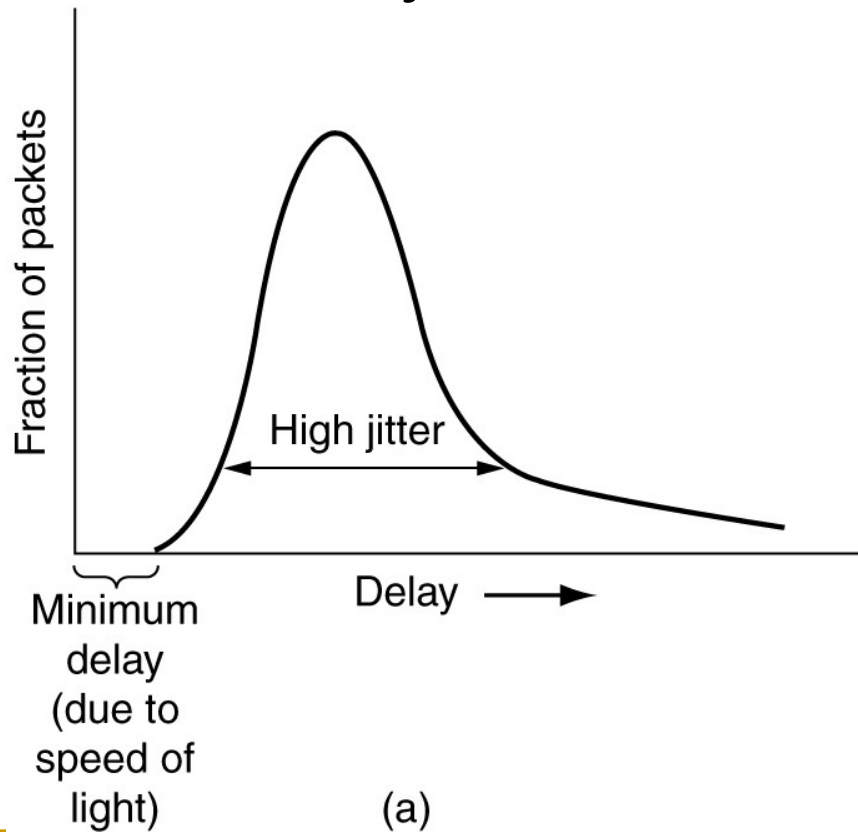
- This is a proactive approach in which the router discards one or more packets *before* the buffer becomes completely full.
- Each time a packet arrives, the RED algorithm computes the average queue length, ***avg***.
- If *avg* is lower than some lower threshold, congestion is assumed to be minimal or non-existent and the packet is queued.

RED, cont.

- If *avg* is greater than some upper threshold, congestion is assumed to be serious and the packet is discarded.
- If *avg* is between the two thresholds, this might indicate the onset of congestion. The probability of congestion is then calculated.

Congestion: jitter control

- Important for audio and video applications?
 - not delay



Congestion: jitter control

- Jitter = variation in packet delay
- Compute feasible mean value for delay
 - compute expected transit time for each hop
 - router checks to see if packet is
 - behind
 - ahead schedule
 - **behind**: forward packet asap
 - **ahead**: hold back packet to get it on schedule again
- Buffering? Depends on characteristics:
 - Video on demand: ok
 - Videoconferencing: nok

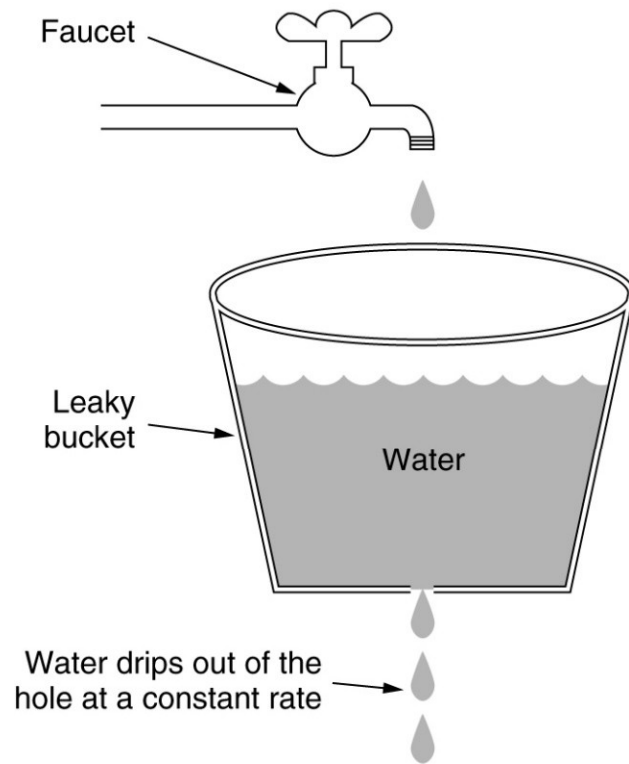
Traffic Shaping

- Another method of congestion control is to “shape” the traffic before it enters the network.
- Traffic shaping controls the *rate* at which packets are sent (not just how many). Used in ATM and Integrated Services networks.
- At connection set-up time, the sender and carrier negotiate a traffic pattern (shape).
- Two traffic shaping algorithms are:
 - Leaky Bucket
 - Token Bucket

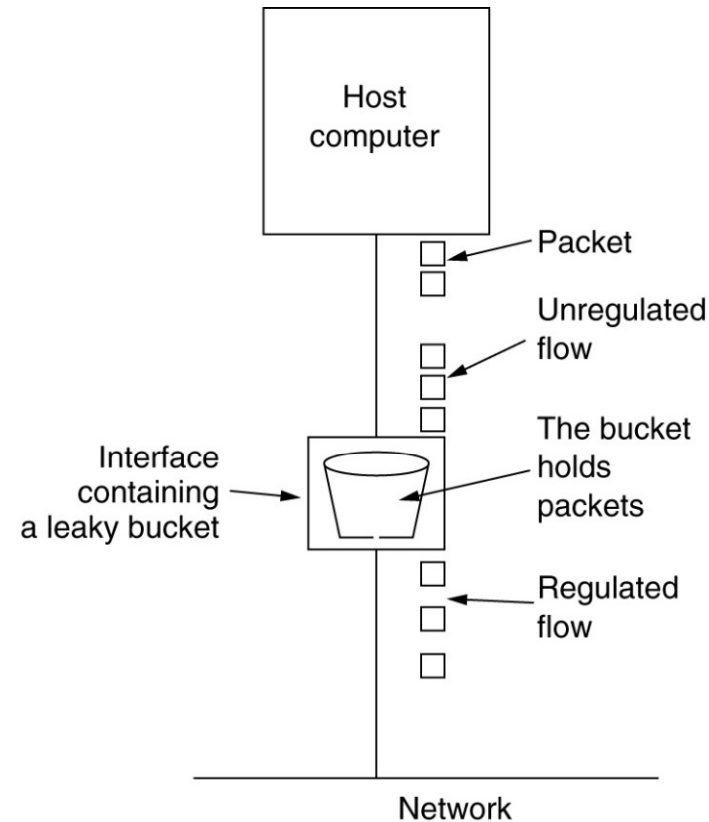
The Leaky Bucket Algorithm

- The **Leaky Bucket Algorithm** used to control rate in a network. It is implemented as a single-server queue with constant service time. If the bucket (buffer) overflows then packets are discarded.
-

The Leaky Bucket Algorithm

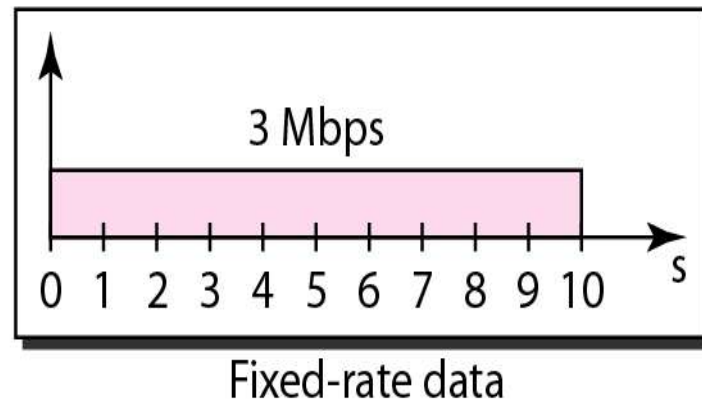
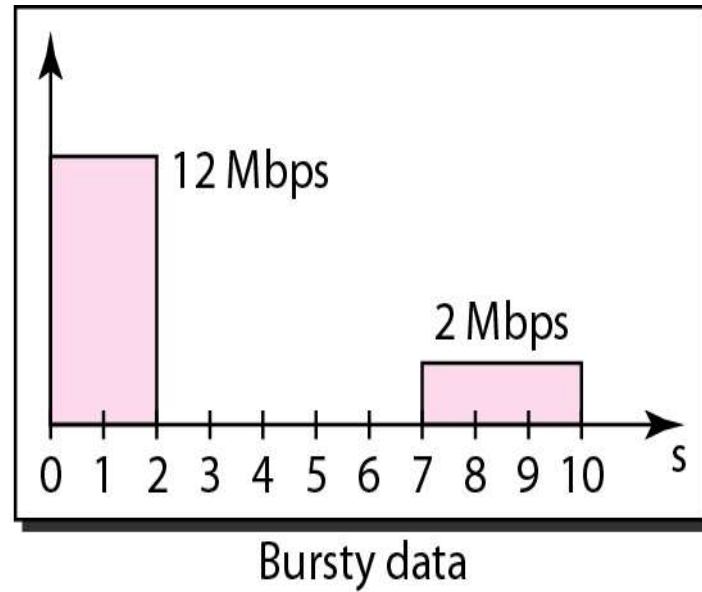
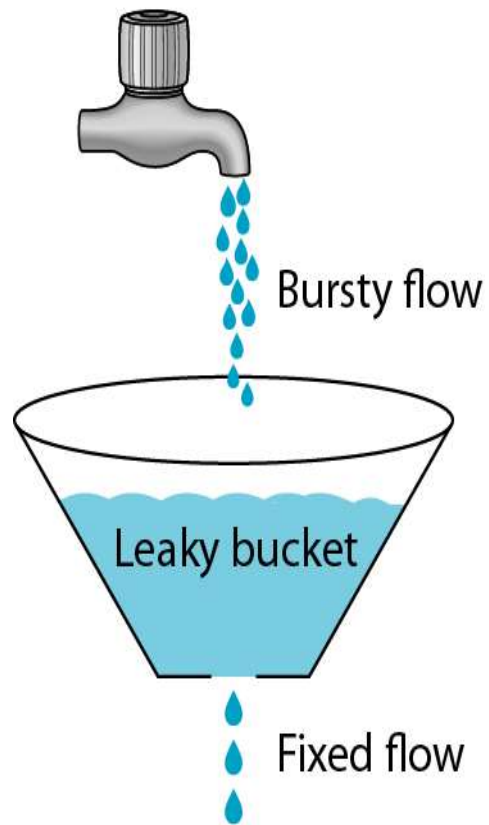


(a)



(b)

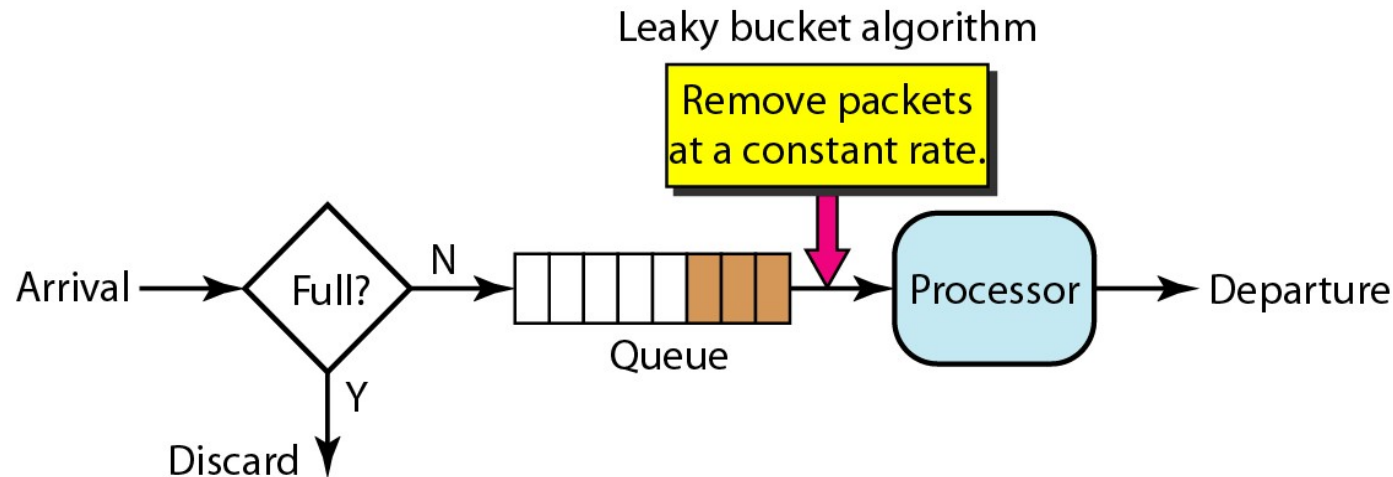
(a) A leaky bucket with water. (b) a leaky bucket with packets.



Leaky Bucket Algorithm, cont.

- The leaky bucket enforces a constant output rate (average rate) regardless of the burstiness of the input. Does nothing when input is idle.
- The host injects one packet per clock tick onto the network. This results in a uniform flow of packets, smoothing out bursts and reducing congestion.
- When packets are the same size (as in ATM cells), the one packet per tick is okay. For variable length packets though, it is better to allow a fixed number of bytes per tick. E.g. 1024 bytes per tick will allow one 1024-byte packet or two 512-byte packets or four 256-byte packets on 1 tick.

Leaky Bucket Implementation



- Algorithm for variable-length packets:

- 1) Initialize a counter to n at the tick of the clock
- 2) If n is greater than the size of the packet, send packet and decrement the counter by the packet size. Repeat this step until n is smaller than the packet size
- 3) Reset the counter and go to step 1

Example

200	700	500	450	400	200
-----	-----	-----	-----	-----	-----

Let $n=1000$ and assume that the queue contains packets of variable size.

1. $n > \text{front of queue}$

$1000 > 200 (\text{true})$

Therefore $n = 1000 - 200 = 800$

Packet size of 200 is sent to the network.

2. Now the Queue is **200 700 500 450 400**

$800 > 400 (\text{front of queue})$ so $800 - 400 = 400$

Packet size of 400 is sent

3. Now the queue becomes 200 700 500 450

$400 < 450$ i.e the front of queue is greater than current 'n' value

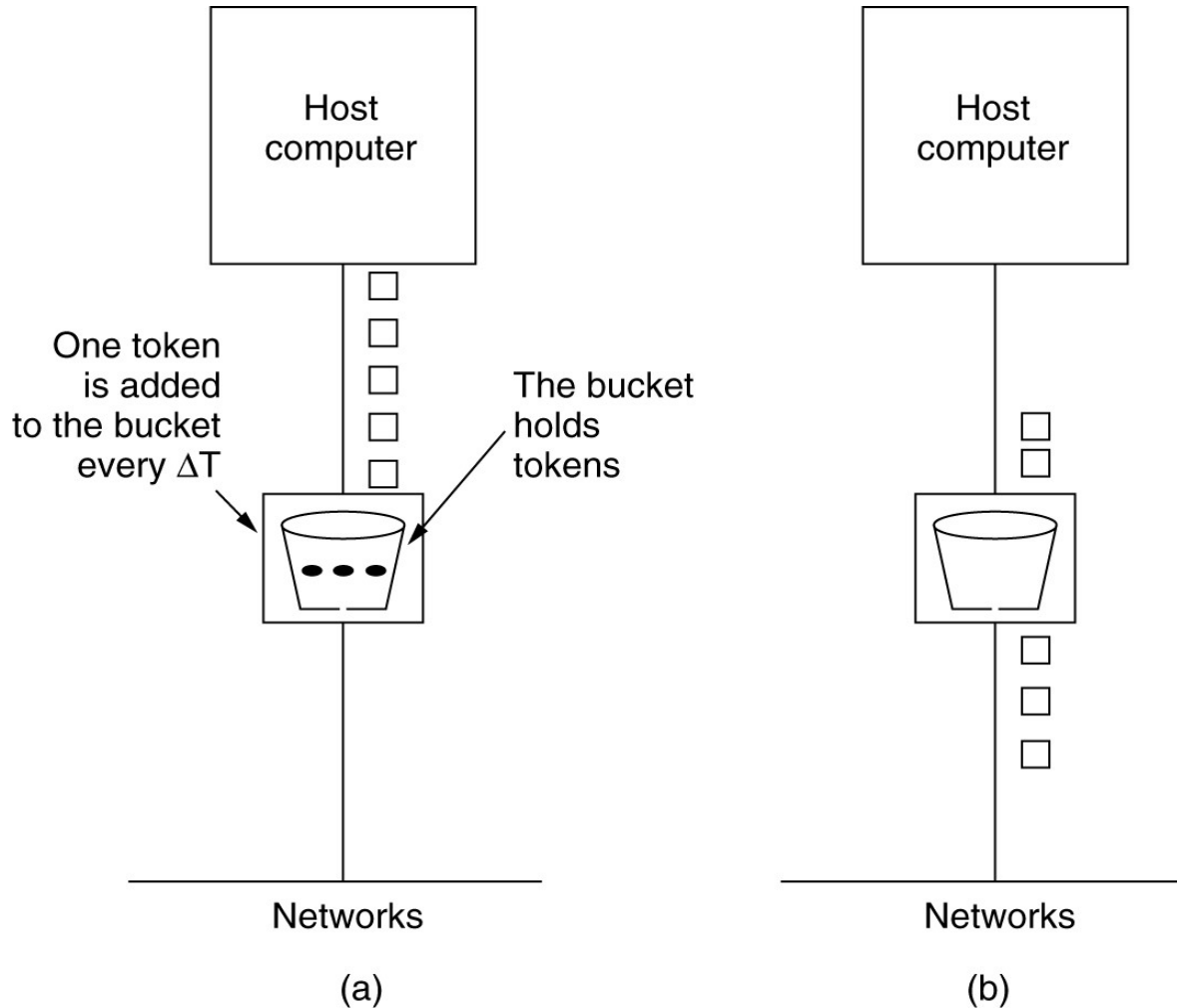
Therefore packet size 450 can not sent. The procedure is stopped and we initialize $n=1000$ on another tick of clock.

This procedure is repeated until all the packets are sent to the network.

Token Bucket Algorithm

- In contrast to the LB, the Token Bucket Algorithm, allows the output rate to vary, depending on the size of the burst.
- In the TB algorithm, the bucket holds tokens. To transmit a packet, the host must capture and destroy one token.
- Tokens are generated by a clock at the rate of one token every Δt sec.
- Idle hosts can capture and save up tokens (up to the max. size of the bucket) in order to send larger bursts later.

The Token Bucket Algorithm

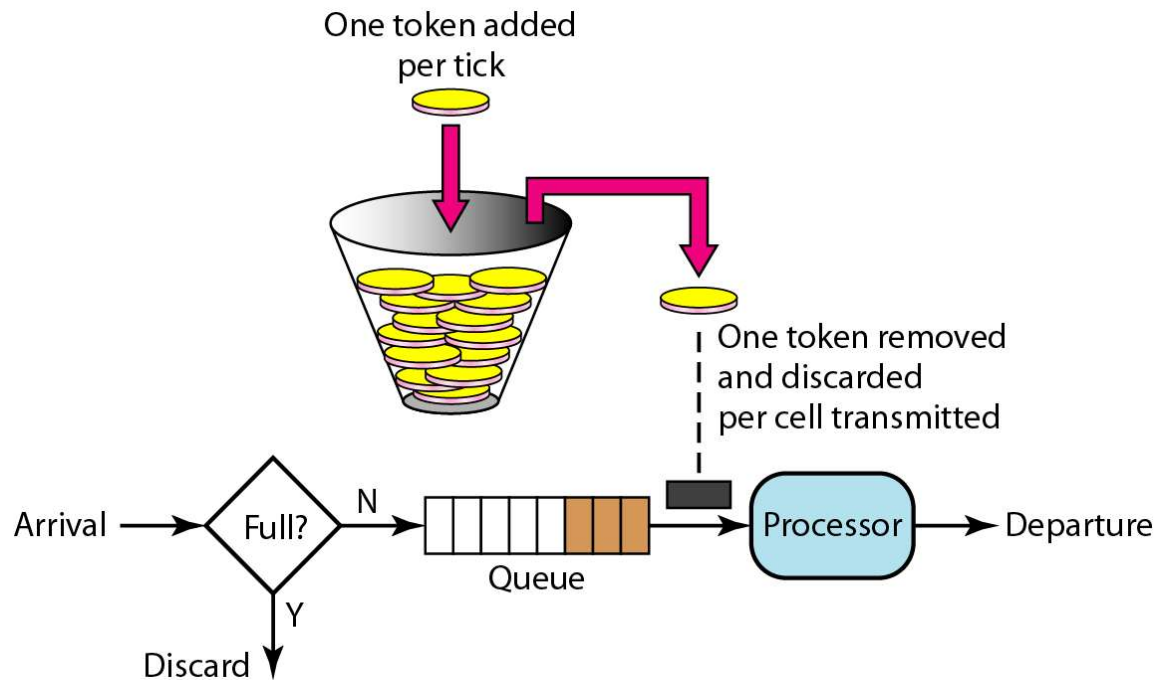


(a) Before.

(b) After.

Token Bucket

- The token bucket allows bursty traffic at a regulated maximum rate.



•

Token Bucket Algorithm:

- A token is added at every Δt time.
- The bucket can hold atmost b-tokens. If a token arrive when the bucket is full it is discarded.
- When a packet has 'm' bytes arrived 'm' tokens are removed from the bucket and the packet is sent to the network
- If less than 'n' tokens are available no tokens are removed from the bucket and the packet is considered to be non-conformant.

The non-conformant packet may be enqueued for subsequent transmission when sufficient tokens have been accumulated in bucket.

Leaky Bucket vs Token Bucket

- LB discards packets; TB does not. TB discards tokens.
 - With TB, a packet can only be transmitted if there are enough tokens to cover its length in bytes.
 - LB sends packets at an average rate. TB allows for large bursts to be sent faster by speeding up the output.
 - TB allows saving up tokens (permissions) to send large bursts. LB does not allow saving.
-

The delivery of a packet to a host or a router requires two levels of addressing: logical and physical. We need to be able to map a logical address to its corresponding physical address and vice versa. This can be done by using either static or dynamic mapping.

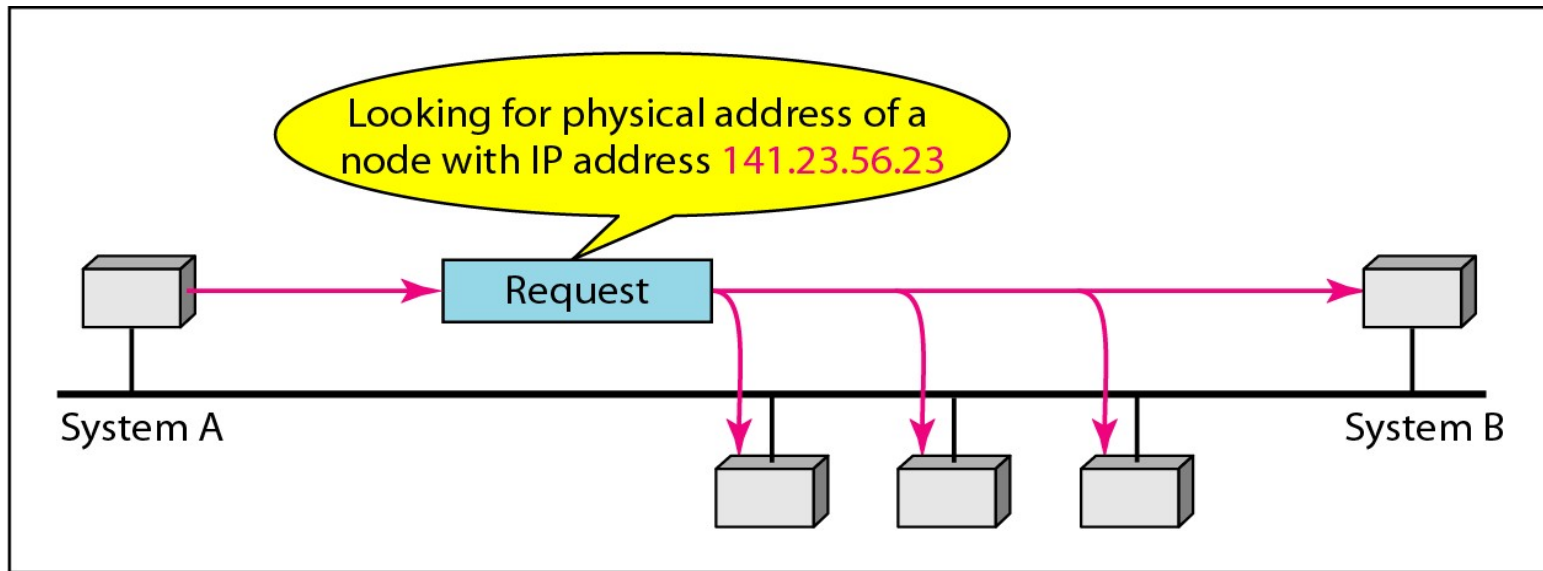
Protocols Used for Mapping

- Address Resolution Protocol (ARP)
- Reverse Address Resolution Protocol (RARP)
- Dynamic Host Configuration Protocol (DHCP)

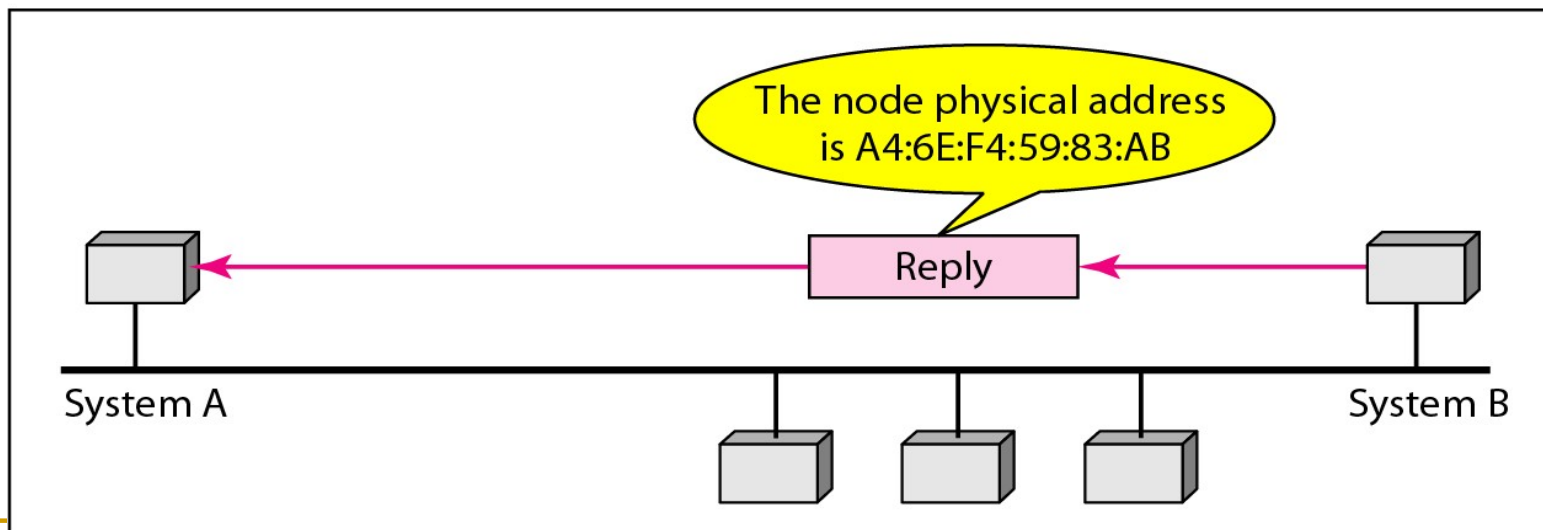
Address Resolution Protocol

- Internet is made of a combination of physical networks connected by internetworking devices such as routers
 - Hosts and routers are recognized at the network level by their logical IP addresses
 - Packets pass through physical networks to reach these hosts and routers
 - At the physical level, the hosts and routers are recognized by their physical addresses
 - ARP is the protocol used to map IP address to MAC address.
-
- View ARP cache (Command prompt): `arp -a`

ARP operation



a. ARP request is broadcast

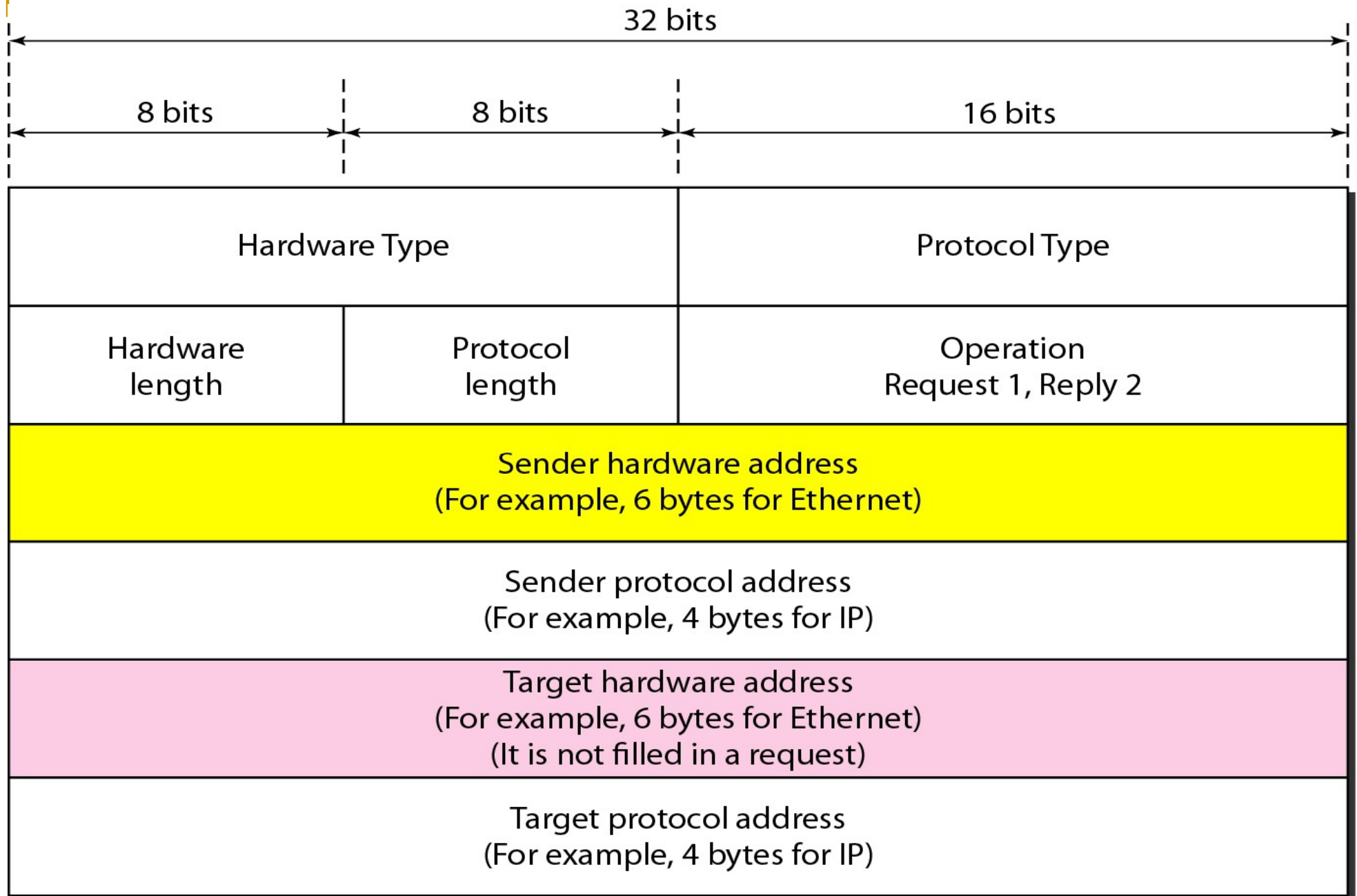


b. ARP reply is unicast

Performance: Cache Memory

- Using ARP is inefficient if system A needs to broadcast an ARP request for each IP packet it needs to send to system B
- ARP can be useful if the ARP reply is cached (kept in cache memory)
- A system that receives ARP reply stores the mapping in cache memory and keep it for 20 to 30 minutes unless there is no space in the cache.

ARP PACKET FORMAT

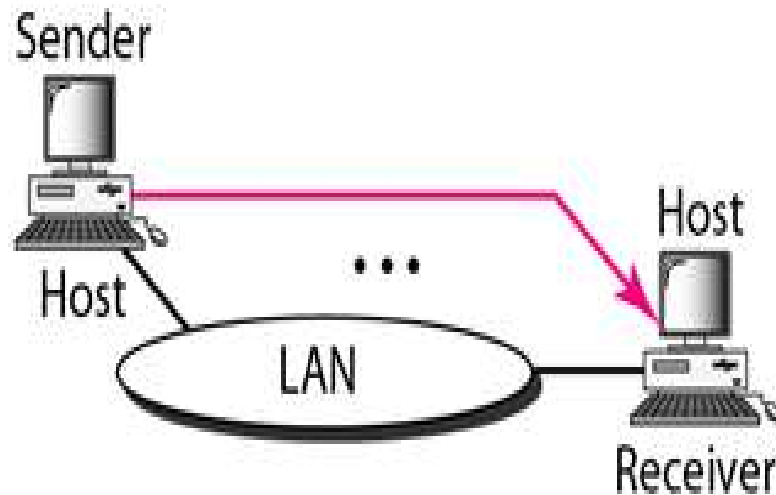


ARP Packet

- **Hardware type:** define the type of the network on which ARP is running. Ethernet is given type 1.
- **Protocol Type:** define the protocol. 0800 for IPv4
- **Hardware length:** define the length of the physical address in bytes. Ethernet the value is 6.
- **Protocol Length:** define the length of the logical address in bytes. For IPv4 protocol the value is 4.
- **Operation:** define the type of packets. ARP request (1) or ARP reply (2).
- **Sender hardware address:** the physical address of the sender.
- **Sender protocol address:** the IP address of the sender
- **Target hardware address:** the physical address of the receiver.
- **Target protocol address:** the IP address of the receiver.

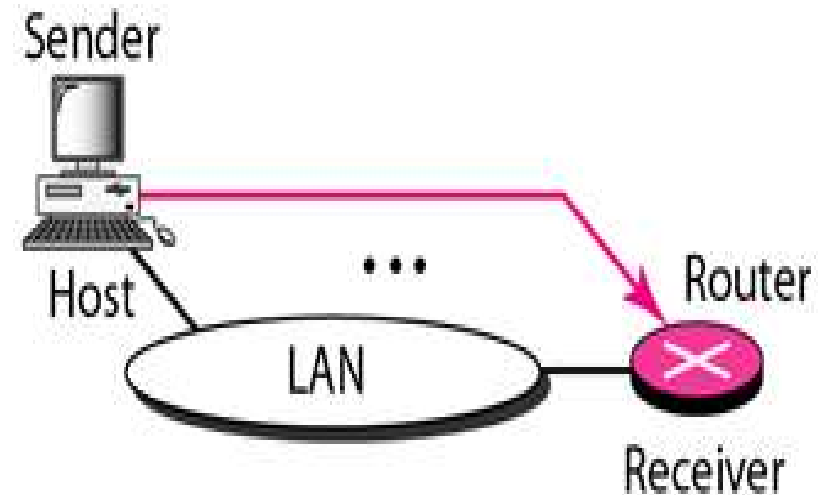
ARP FOUR CASES

Target IP address:
Destination address in the IP datagram



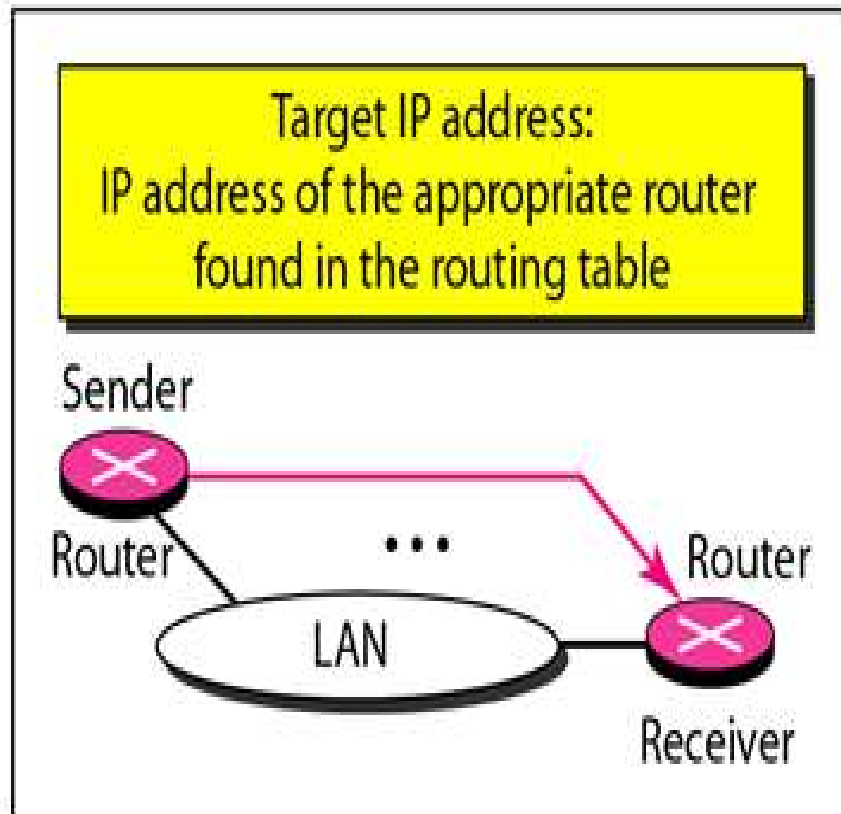
Case 1. A host has a packet to send to another host on the same network.

Target IP address:
IP address of a router

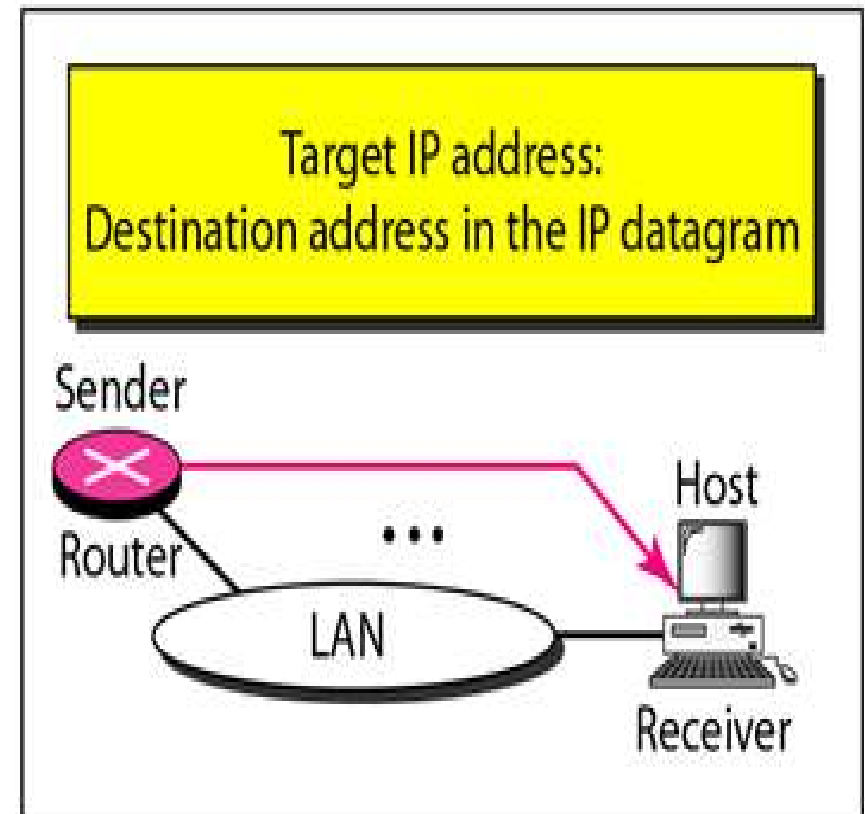


Case 2. A host wants to send a packet to another host on another network. It must first be delivered to a router.

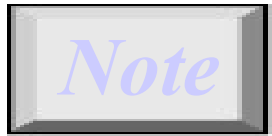
ARP FOUR CASES



Case 3. A router receives a packet to be sent to a host on another network. It must first be delivered to the appropriate router.



Case 4. A router receives a packet to be sent to a host on the same network.

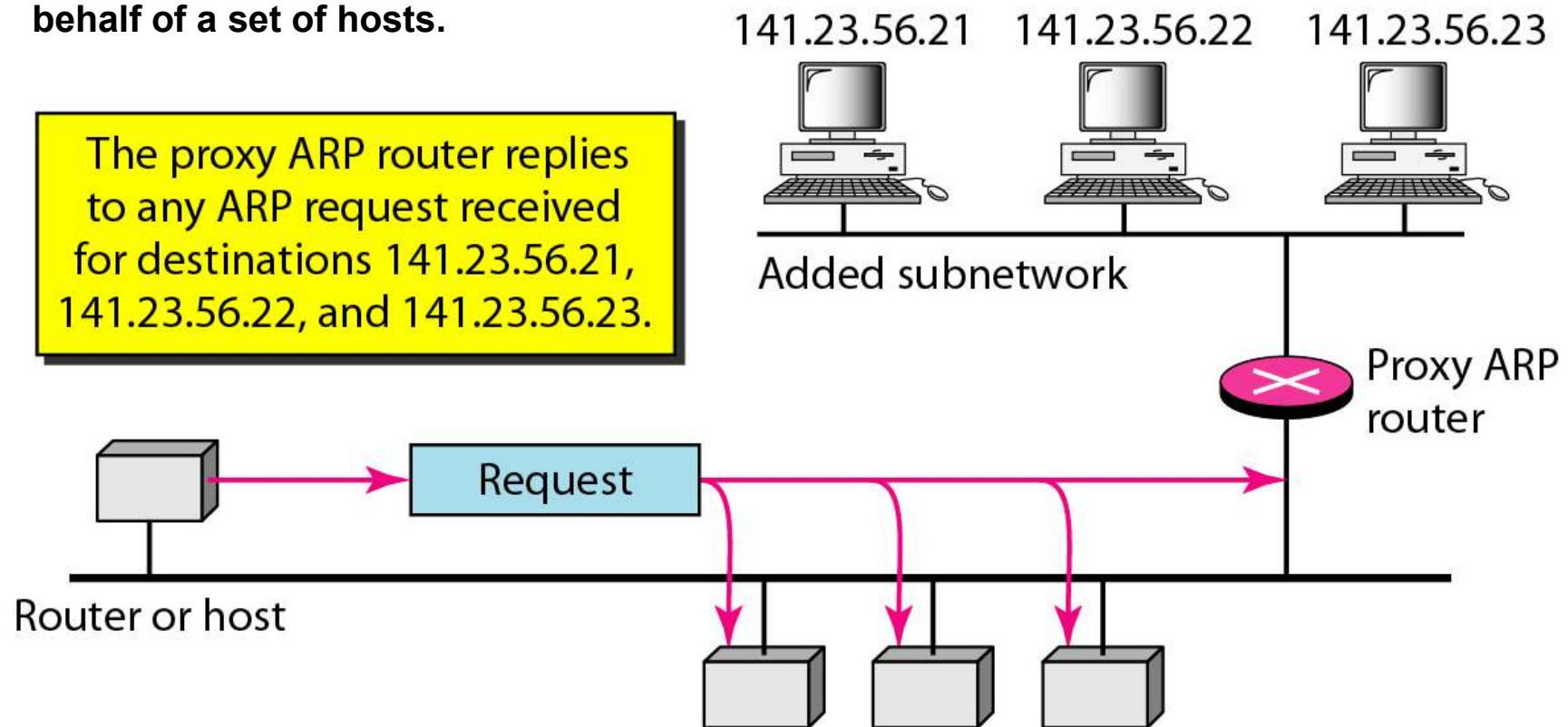


**An ARP request is broadcast;
an ARP reply is unicast.**

Figure 21.6 *Proxy ARP*

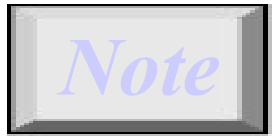
A proxy ARP is an ARP that acts on behalf of a set of hosts.

The proxy ARP router replies to any ARP request received for destinations 141.23.56.21, 141.23.56.22, and 141.23.56.23.



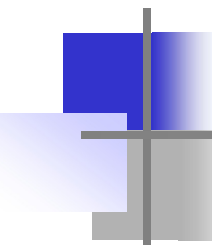
Reverse Address Resolution Protocol (RARP)

- Finds the logical address for a machine that knows only its physical address.
- To create an IP datagram, a host or a router needs to know its own IP address or addresses.
- A RARP request is created and broadcast on the local network.
- The requesting machine must be running a RARP client program; the responding machine must be running a RARP server program.
- There is a serious problem with RARP: Broadcasting is done at the data link layer.
- This means that if an administrator has several networks or several subnets, it needs to assign a RARP server for each network or subnet.



DHCP provides static and dynamic address allocation that can be manual or automatic.

- **Static Address Allocation:** In this capacity DHCP acts as BOOTP does. So, A host running the BOOTP client can request a static address from a DHCP server. A DHCP server has a database that statically binds physical addresses to IP addresses.
- **Dynamic Address Allocation:** DHCP has a second database with a pool of available IP addresses. When a DHCP client requests a temporary IP address, the DHCP server goes to the pool of available (unused) IP address and assigns an IP address for a negotiable period of time.
- **Command prompt:** `ipconfig/all`, `ipconfig/release`, `ipcnfig/renew`.



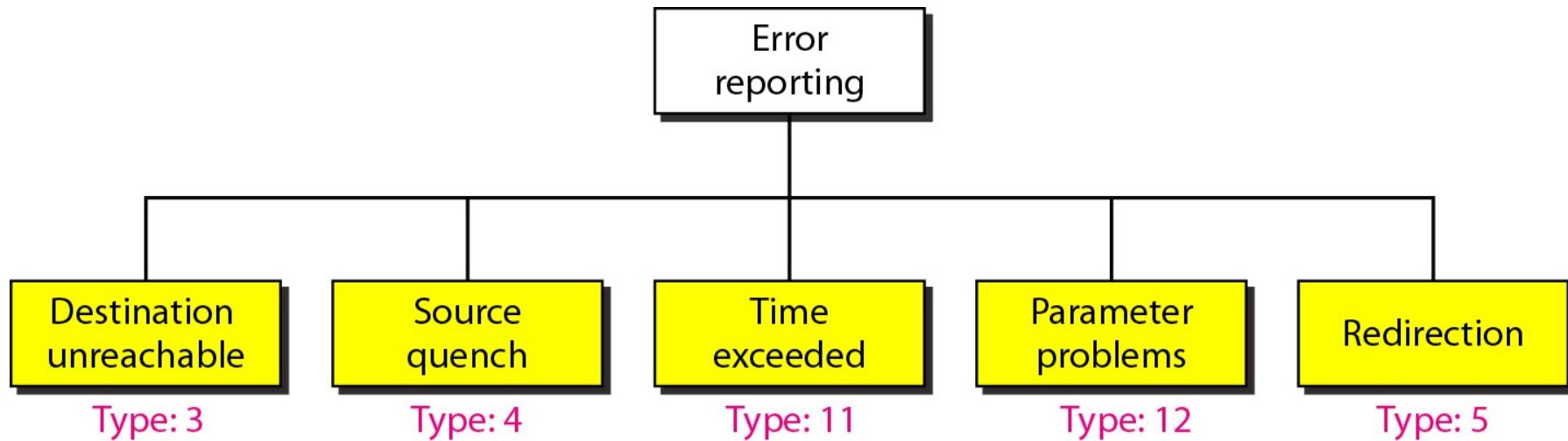
Internet Control Message Protocol

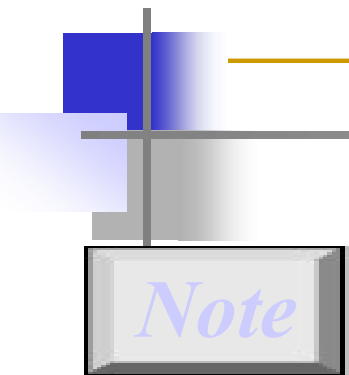
ICMP always reports error messages to the original source.

Types of Messages

- 1. Error reporting messages**
- 2. Query messages**

Figure 21.9 *Error-reporting messages*





Note

Important points about ICMP error messages:

- ☐ No ICMP error message will be generated in response to a datagram carrying an ICMP error message.
- ☐ No ICMP error message will be generated for a fragmented datagram that is not the first fragment.
- ☐ No ICMP error message will be generated for a datagram having a multicast address.
- ☐ No ICMP error message will be generated for a datagram having a special address such as 127.0.0.0 or 0.0.0.0.

Figure 21.11 *Redirection concept*

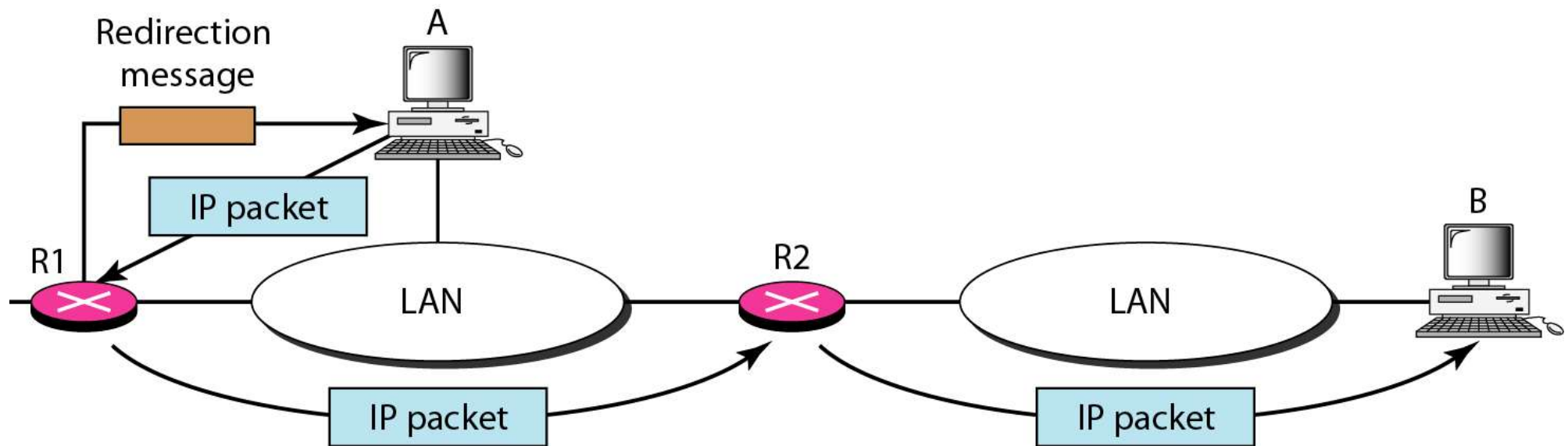


Figure 21.12 *Query messages*

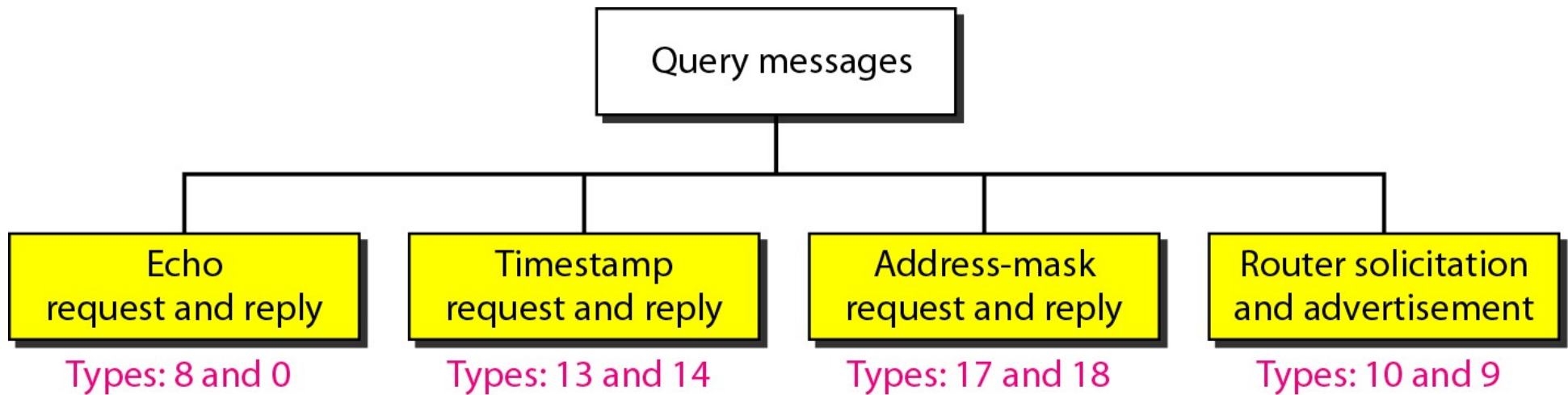


Figure 21.13 *Encapsulation of ICMP query messages*



De-bugging tools:

1.Ping

2.tracert